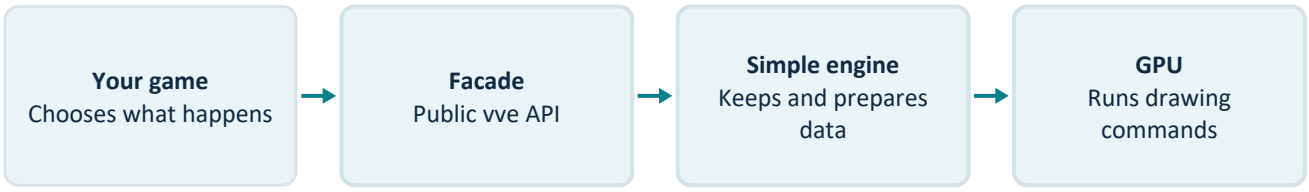
Vienna Vulkan Engine, explained

The facade classes, the simple engine, and a comprehensive game programmer guide.

This guide explains the current Vienna Vulkan Engine source. It follows one textured object from a model file, through normal computer memory, into GPU resources, and finally onto the screen.



What you will understand

A source-grounded book in three parts: a full introduction to the public facade; a walkthrough of the simple engine's initialization, assets, resources and drawing; and a practical guide to building games with the API.

How to read it

Start with the reading map on page 2. Each part has linked chapter contents. The facade starts on page 3, the simple engine on page 23, and the game guide on page 51. Source maps follow the three parts.

> **The most useful distinction**
> A model file, a CPU scene and a GPU resource are three different things. Loading a file does not automatically display it. Creating a render object does not necessarily upload it immediately.

Source snapshot: commit 4f0270d3739d (clean working tree).
Module: VVEngine. **Public namespace:** vve.
Implementation: vve::simple. Scene hierarchy pages updated for 4b54ba1a.

Source: [S01], [S03] | Maps: 76-78

Reading map and contents

Read in order: interface, implementation, then practical game programming.

Page	Chapter
3	Part I: the facade classes
23	Part II: the simple engine
51	Part III: the game programmer guide
76	Source map and stepping locations
77	Game guide source references
78	Facade source references

Three parts, three questions

Part I: What can my game call? Learn the public classes, how an engine is created, what World returns, and the meanings of handles and descriptors.

Part II: What happens inside? Follow the simple implementation from platform startup to model import, CPU storage, GPU allocation, frame recording and presentation.

Part III: How do I make a game? Work through a complete starter and practical recipes for movement, input, models, components, lighting, GUI, effects and levels.

> **Choose a useful reading route**
> For a first project, read the facade introduction and then the starter on pages 54-55. For debugging, follow the simple-engine frame walkthrough and use the source maps on pages 76-78.

Part I: the facade classes

The complete public-class introduction: roles, construction, stored state and API contracts.

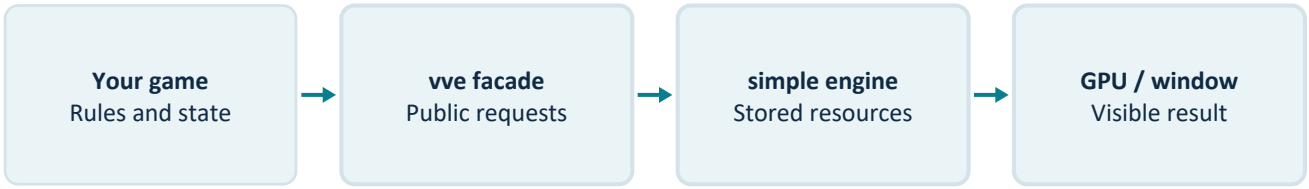
Page	Chapter
4	What the facade gives you
5	Facade classes: who provides API calls?
6	Engine: the owner and coordinator
7	All public ways to create an engine
8	Engine creation, one step at a time
9	World: references to the stored objects
10	UserSystems and automatic hooks
11	WindowSetup and WindowSetups
12	WindowSystem and Window
13	InputState and DefaultCameraController
14	AssetSystem: source data and queries
15	RenderSystem: visible objects
16	RenderSystem: imported scenes and lifetime
17	RenderSystem: views, lights and observation
18	GuiSystem and post-processing callbacks
19	BasicECS and ECS: your component store
20	Typed handles and small value helpers
21	Shared data types: a complete reading map
22	Errors, files and the public boundary

Begin with the class map, then follow the Engine constructor and World references. Window, asset and render API chapters show what each service stores or returns. The final chapters explain identities, descriptors and the module/file boundary.

What the facade gives you

A single public interface connects your game to the selected engine.

A facade is the set of classes and functions your game calls. In Vienna Vulkan Engine, start with **import VVEngine;** and use names in **namespace vve**. The module name is VVEngine; the central C++ class is vve::Engine. They are different kinds of names.



Say what you want to happen

Call addTexturedPlane() to make ground, setObjectTransform() to move an object, or setCamera() to choose a view. The facade forwards each request to the implementation. The implementation decides how to store geometry, upload pixels, allocate Vulkan resources and draw the result.

The interface also supplies a shared vocabulary

Transform, Position, Camera, typed handles and Error describe requests and results. They carry useful meaning without exposing the internal Catalog, RenderScene or Vulkan device. The ECS is shared public storage for your own component types.

Layer	Names you will encounter
Game-facing interface	vve::Engine, World, AssetSystem, RenderSystem, WindowSystem, GuiSystem, ECS
Selected implementation	vve::simple::Engine and its window, asset, GUI and rendering classes
Implementation details	Catalog, RenderMesh, ForwardRenderer, WindowTarget and Vulkan resource owners

How to read this book
Part I explains the public classes and their contracts. Part II follows the simple engine behind those contracts. Part III shows how to build a game with them. Implementation-specific behavior is identified where it matters.

Source: [S01], [S02], [S32], [S37], [S38] | Maps: 76-78

Facade classes: who provides API calls?

The public layer contains service APIs, construction helpers and data types.

Class / template	API role and state
Engine<TSystems...>	API: lifecycle, world lookup and frame context. Owns one implementation and the user systems.
EngineBuilder<TSystems...>	Construction API: collects startup options and builds an Engine.
World<TObjects...>	Access API: get<T>() returns a reference to an existing service or user system.
AssetSystem	Service API: imports and queries source scene, node, mesh, material, light and camera data.
RenderSystem	Service API: creates visible objects; controls placement, cameras, lighting, effects and capture.
WindowSystem	Service API: lists/finds windows and hands out input access.
Window	Read-only API: handle, identity, extent, renderer and current platform state.
InputState	Input API: key and per-window mouse queries; also exposes input-state mutation methods.
GuiSystem	Service API: registers the GUI draw callback.
BasicECS<TTraits> / ECS	Storage API: creates entities and adds, reads, writes, removes or selects components.
DefaultCameraController	Utility API: turns keyboard input and elapsed time into a Camera.
WindowSetup / WindowSetups	Configuration API: fluent window settings and a startup window collection.
UserSystems<T...>	Data: tuple of user-system values supplied to Engine.
MakeUserSystems / makeUserSystems	Utility API: builds a correctly typed UserSystems bundle from values.
DefaultECSTraits	Configuration type: selects ECS storage behavior; slot maps are not implemented here.
TypedHandle<Tag> / HandleHash	Value/utility API: typed identity queries and hashing; no resource ownership.
Camera and descriptor structs	Data, with a few convenience helpers such as Camera::lookAt().

An API method can read state as well as change it. A struct with fields is also part of the public contract, but filling in that struct does not create a window, object or GPU resource until you pass it to the appropriate service.

Source: [S01], [S32], [S36], [S37], [S38], [S39], [S41], [S50] | Maps: 76-78

Engine: the owner and coordinator

The facade Engine keeps one implementation and your registered systems alive.

Engine<TSystems...> combines engine services with the concrete types of your game systems. Engine<> has no registered game systems. Engine<Movement, Score> owns one Movement and one Score in a tuple. The template parameters describe the game systems; they do not choose the rendering implementation.

Public call	Meaning
init()	Initialize the implementation, then run optional user init hooks. Return expected<void,Error>.
step()	Poll/update backend state, call user updates, render, then return running/stopped or an error.
run()	Call init and repeat step until stopped or an error occurs.
world()	Create a small reference view of the existing services and user systems.
frameContext()	Read the latest polled frame index and delta time.
versionMajor() / versionName()	Read implementation version information.

What is stored inside

A unique_ptr owns the selected EngineImpl. The facade stores references to its ECS and lightweight wrappers for assets, GUI, windows and rendering. It separately owns your user-system tuple and remembers how many init hooks completed successfully.

Keep the engine at a stable address

Engine cannot be copied or moved. Its wrapper references must continue to refer to the same underlying subsystem objects. The builder returns a newly constructed Engine directly, using C++ guaranteed copy elision; it does not move an already existing engine.

> **Construction is not full initialization**
> Constructing Engine creates the C++ owners and connections. Call init() before step(). In the simple implementation, platform/windows initialize first and the renderer creates GPU resources lazily when rendering starts.

Source: [S01], [S03] | Maps: 76-78

All public ways to create an engine

Choose default construction, typed options or a fluent builder.

```
import std;
import VVEngine;

vve::Engine<> defaults;

vve::Engine<> configured{
    vve::ApplicationName{.value = "My game"},
    vve::MaxFrames{.value = vve::FrameCount{.value = 120}}
};

auto built = vve::EngineBuilder<>{}
    .applicationName("My game")
    .addWindow(vve::WindowSetup{}
        .id("main").title("My game")
        .extent({.width = 1280, .height = 720}))
    .build();
```

Choice	What to supply
Defaults	Engine<> or Engine<YourSystem...> with default-constructible systems.
Direct typed options	ApplicationName, MaxFrames, WindowSetups and a matching UserSystems bundle.
Builder	applicationName(string), maxFrames(MaxFrames), windows(WindowSetups), addWindow(WindowSetup), userSystems(UserSystems).
Explicit system values	Engine<Game>{makeUserSystems(Game{...})}; supply the same system types and order as Engine.

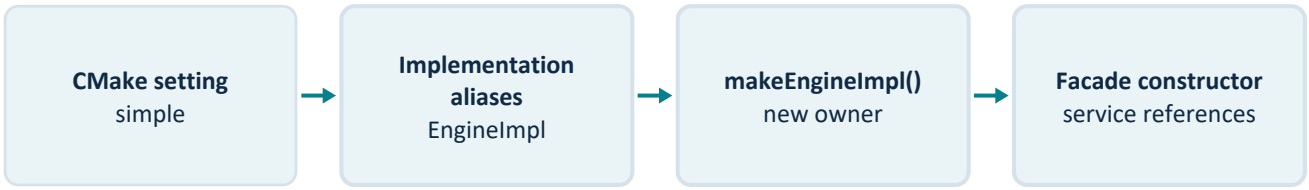
These are alternative construction examples, not a recommendation to create three engines in one application. They all reach the same factory and implementation. An unknown direct option type causes a compile error. Repeated name/frame/window options are processed in order, with the later value replacing the earlier value.

> **MaxFrames is a step limit**
> A value of zero means no configured frame cap. This option does not set frames per second. Use run() for engine-controlled execution, or init() and step() when your application owns the loop.

Source: [S01], [S29], [S36] | Maps: 76-78

Engine creation, one step at a time

CMake selects a concrete type; the factory allocates an object of that type.



1. Configure the build. VVE_ENGINE_IMPLEMENTATION_NAMESPACE selects src/implementations/<name>.ixx and the corresponding engine source folder. The current repository supplies only simple. Selecting a new name requires an actual compatible implementation and adapter; changing the setting alone cannot create one.

2. Compile the adapter. The simple adapter defines detail::EngineImpl as simple::Engine, and matching aliases for every wrapped subsystem. An alias names the concrete type. It is not an allocated object and it is not a runtime plugin selector.

3. Collect options. The facade extracts ApplicationName, MaxFrames and optional window descriptors. UserSystems are retained for the later system-tuple setup rather than passed into the backend factory.

4. Allocate the implementation. Inline detail::makeEngineImpl() calls make_unique<EngineImpl>. With explicit windows it also passes WindowsImpl containing their descriptors; otherwise it uses the constructor without that argument. The resulting unique_ptr is the facade owner.

5. Bind services and systems. The facade obtains impl_->ecs(), assets(), gui(), windowSystem() and renderSystem(). It constructs wrappers referring to those objects. Then it copies/moves the supplied user-system bundle or default-constructs the systems.

```
// The selected adapter establishes the type:
using EngineImpl = simple::Engine;

// The factory creates an object of that type:
return std::make_unique<EngineImpl>(name, max_frames);
```

How a call finds the real state

The alias tells the compiler which methods exist. The stored object supplies the state those methods operate on. A RenderSystem wrapper holds a reference to a real RenderSystemImpl, and forwards calls to that instance. There is no virtual dispatch or runtime engine switch in this design.

Source: [S01], [S02], [S53], [S54] | Maps: 76-78

World: references to the stored objects

get() finds an existing object by its exact C++ type.

```
auto world = engine.world();
auto& render = world.get<vve::RenderSystem>();
auto& assets = world.get<vve::AssetSystem>();
auto& windows = world.get<vve::WindowSystem>();
auto& gui = world.get<vve::GuiSystem>();
auto& ecs = world.get<vve::ECS>();
// With Engine<Game>, also: world.get<Game>()
```

World<TObjects...> contains a tuple of reference_wrapper values. Engine creates the view from references to its ECS, public subsystem wrappers and user-system objects. There is no separate World implementation and no second scene stored inside World.

Type lookup and object access are separate jobs

get<RenderSystem>() searches the tuple at compile time for that exact target type. It then returns the stored reference. Calling render.setCamera() uses the wrapper object you obtained; the wrapper forwards to its stored implementation reference. The type determines the method, and the object supplies its actual state.

A missing type is a compile error. If a type appears more than once, get<T>() finds the first one, so use distinct system types when you need to access them individually. In templated hooks, write world.template get<T>() because the World type depends on a template parameter.

Lifetime and constness

A World can be move-constructed, but cannot be copied or assigned. It is cheap to obtain another one from engine.world(). Its references remain usable only while the engine and the referred objects live. Making the view const does not freeze the objects referred to by its reference wrappers.

> **Copies of subsystem wrappers are still views**
> AssetSystem, RenderSystem, WindowSystem, GuiSystem, InputState and Window allow copy/move construction but delete assignment. Copying one does not duplicate an asset catalog or GPU scene. Window is read-only; the other wrappers retain access to the same implementation state.

Source: [S01], [S32], [S36], [S37], [S38], [S41] | Maps: 76-78

UserSystems and automatic hooks

The engine calls ordinary methods on the system values it owns.

```
struct Game {
    int score = 0;
    template<class W> void init(W& world) {
        // Prepare game state through public services.
    }
    template<class W>
    void update(W& world, const vve::FrameContext& frame) {
        // Read input, advance game state, publish transforms.
    }
};
vve::Engine<Game> engine{vve::makeUserSystems(Game{})};
```

UserSystems<T...> is a tuple of system values. The callable object makeUserSystems (type MakeUserSystems) deduces concrete types and forwards the supplied values into that tuple. No base class, virtual method or registration name is required.

Available hook	When / priority
init(world)	Optional, called after backend init, in tuple order.
update(world, frame, window_frame)	Highest update priority; receives current copied WindowFrameData.
update(world, frame)	Next priority; receives current FrameContext.
update(world)	Next priority; needs no frame arguments.
No matching hook	No call for that stage.

Only the highest matching update overload is called for a system. Hooks run in tuple order on the engine-owned values, not on an original object that was copied into a bundle. Window snapshots are prepared only if at least one system accepts the three-argument update.

A hook may return void or std::expected<void,Error>. Only that exact expected type propagates failure; other return values are discarded. A failing init is retried on the next init() while earlier successful hooks stay complete. A failing update stops later systems and rendering for that step.

> **Copies, moves and builders**
> Direct construction with a temporary bundle can move system values, including move-only state. EngineBuilder::build() is const and copies its stored bundle. A system without a default constructor needs an explicitly supplied bundle; use direct construction when builder default storage or copying does not suit the type.

Source: [S01], [S32] | Maps: 76-78

WindowSetup and WindowSetups

These helpers describe windows before initialization creates them.

```
vve::WindowSetups windows{
    vve::WindowSetup{}.id("main").title("Player")
        .extent({.width = 1280, .height = 720})
        .resizable(true).visible(true),
    vve::WindowSetup{}.id("map").title("Map")
        .extent({.width = 640, .height = 480})
        .renderer(vve::RendererId{.value = "forward"})
};
vve::Engine<> engine{std::move(windows)};
```

Setting	Meaning / default
id(string)	Application-local lookup name. Default main; must be nonempty and unique.
title(string)	Visible title. Default VVE simple; the simple engine substitutes the application name for the default title.
extent(PixelExtent)	Initial window-coordinate size, default 960 x 540; nonzero dimensions required.
position(x,y)	Optional screen position; otherwise leave platform placement unspecified.
renderer(RendererId)	Empty or forward draws; none opts out. Unknown identifiers fail init.
resizable(bool) / visible(bool)	Both default true. Hidden windows can still render for captures/tests.
WindowSetups::add(WindowSetup)	Append a startup descriptor. An empty collection gets a default main window.

WindowSetup owns one WindowDesc. WindowSetups owns the collection. Fluent methods return a reference to the same helper so you can chain settings; they do not immediately contact SDL or create a native window.

Startup size versus drawable size
Startup extent uses window coordinates. Window::extent() later reports drawable pixels, which can differ on a scaled display. The renderer uses drawable pixels for its swapchain and aspect ratio. These startup helpers do not expose runtime window creation or resizing commands.

Source: [S01], [S29], [S36], [S03], [S04] | Maps: 76-78

WindowSystem and Window

The manager returns small read-only views of the windows it owns.

```
auto world = engine.world();
auto& windows = world.get<vve::WindowSystem>();
auto main = windows.findWindow("main");
if (main) {
    auto handle = main->handle();
    auto drawable_size = main->extent();
    // Save the handle for camera, mouse and capture APIs.
}
```

WindowSystem call	Result
name()	Window backend name as a string_view.
windowCount()	Number of windows in the manager.
windows()	Vector of Window wrappers; wrappers still refer to owned windows.
findWindow(id) / findWindow(handle)	optional<Window>; empty when lookup fails.
input()	InputState wrapper over the same live input object.

Window call	What it reads
handle(), id(), title()	Typed runtime identity, stable application ID and platform title.
extent(), rendererId()	Drawable pixel dimensions and renderer selection.
focused(), minimized(), shouldClose()	State from platform events.

Window stores a const implementation reference. It has no public setTitle, resize or destroy call. WindowSystem owns window management; a Window value gives you access to state, not ownership of the native resource. string_view results also borrow the underlying strings.

Current simple-engine behavior

A close request on any managed window stops the engine. A minimized or zero-size target can skip drawing; a hidden target can still draw. Each rendering window has its own swapchain and aspect ratio. Cameras are controlled through RenderSystem, not Window or ECS camera bindings.

> **Live view versus snapshot**
> Reading Window after event polling sees the refreshed state. WindowFrameData, supplied to an optional three-argument system hook, owns copies of WindowInfo for that frame. Keeping a Window wrapper is not the same as keeping a historical snapshot.

Source: [S36], [S29], [S03], [S04] | Maps: 76-78

InputState and DefaultCameraController

Use frame edges for actions and held state for continuous movement.

InputState query	Meaning
isKeyDown(Key or keycode)	Held now; useful for walking or continuous movement.
wasKeyPressed(Key or keycode)	Press edge in the current poll; useful for one action.
wasKeyReleased(Key or keycode)	Release edge in the current poll.
mousePosition(WindowHandle)	Optional normalized position: top-left (0,0), bottom-right (1,1), y down.
mouseDelta(WindowHandle)	Frame movement in the same normalized units.
mouseWheelDelta(WindowHandle)	Scroll movement in ticks.

InputState is a wrapper over live state. A saved wrapper sees future polls; it does not preserve an old frame. Keyboard state is shared, while mouse data is keyed by window. Drag coordinates can leave 0..1. For mouse look, scale normalized horizontal movement by the window aspect ratio.

beginFrame(), holdKey(), pressKey(), releaseKey(), setMousePosition(), addMouseDelta() and addMouseWheelDelta() are also public. Normally the engine owns event polling and frame resets. Calling these manually during gameplay changes the same state your other systems read.

A ready-made keyboard camera

```
// Keep controller as a member of your game system.
auto input = world.template get<vve::WindowSystem>().input();
auto camera = controller.update(input, frame.delta_time);
world.template get<vve::RenderSystem>().setCamera(camera);
```

DefaultCameraController owns eye, yaw, pitch, move_speed, turn_speed and max_pitch. WASD moves, Q/E moves vertically, arrows turn, and Shift doubles movement speed. Its default eye is (0,6,9), movement speed 4.8 and turn speed 1.5. Looking up changes the forward movement direction, so this is a flying camera rather than a ground-constrained player controller.

> **Input timing**
> Registered update hooks run after polling and before drawing. Code before engine.step() reads the previous poll. GUI capture can consume keyboard/mouse events, while releases still reach the game to avoid stuck held keys. The built-in controller does not implement mouse look.

Source: [S36], [S40], [S04], [S01] | Maps: 76-78

AssetSystem: source data and queries

Importing a scene makes source records available; rendering is a separate request.

```
auto& assets = world.get<vve::AssetSystem>();
auto source = assets.loadScene("assets/model.glb");
if (!source) { return std::unexpected(source.error()); }
auto nodes = assets.sceneNodes(*source);
auto root = assets.sceneRootNode(*source);
```

API family	Calls and purpose
Create/import	loadScene(path) imports file data. addScene(ObjectName) creates an empty source scene. containsScene tests membership.
Scene contents	sceneName; sceneNode/Mesh/Material/Texture/Light/CameraCount; sceneRootNode; sceneNodes/Meshes/Materials/Lights/Cameras.
Hierarchy	sceneNodeChildren(scene,node) and sceneNodeParent(scene,node); parent is optional inside an expected result.
Nodes	nodeName, nodeTransform, nodeMeshes, nodeMaterials.
Meshes	meshName, meshVertexCount, meshIndexCount, meshMaterial, meshBounds; meshPositions, meshNormals, meshTexcoords, meshIndices.
Other source records	materialName, lightData and cameraData.

Returned vectors and descriptors are owning values. They remain independent of later catalog growth, but reading large mesh arrays repeatedly is expensive. Keep handles and query detailed geometry when loading or inspecting a level, rather than every frame.

A node transform is relative to its parent. A mesh holds reusable geometry; a node can attach meshes and materials. sceneNodes follows imported tree creation order. sceneTextureCount counts distinct texture source paths used by source materials, not the number of live render texture slots.

Source handles do not draw
Keep the SceneHandle and pass it to RenderSystem::instantiateScene(). addScene() alone does not build procedural geometry. There is no public AssetSystem scene-unload API; RenderSystem removal/purging manages rendering caches, while imported catalog data remains owned by the engine.

Source: [S38], [S05], [S09] | Maps: 76-78

RenderSystem: visible objects

Creation returns an object handle; later calls address that placed object.

Creation API	What you supply
addPlane	Half extent, linear color and optional Transform; also accepts PlaneDescriptor.
addCuboid	Local minimum/maximum corners, color and Transform; also CuboidDescriptor.
addTriangleMesh	Position vector, triangle index vector, color and Transform.
addTexturedPlane	Half extent, texture path, UV scale and Transform.
addTexturedCuboid	Minimum/maximum, texture path and Transform; also TexturedCuboidDescriptor.

These calls return expected<RenderObjectHandle,Error>. Save the handle after checking success. A RenderObjectHandle identifies one visible placement; it is not an ECS Entity or an imported MeshHandle. Geometry and materials may be shared by multiple placements inside the simple engine.

Object API	Purpose
setObjectTransform / objectTransform	Set or read the placement transform.
setObjectVisible / objectVisible	Hide/show an object or read its visibility.
setObjectCastsShadow	Choose whether the object participates in shadow drawing.
setObjectUnlit	Choose whether lighting affects its displayed surface.
setObjectMeshPositions	Replace positions with a valid same-size vector for this object.
removeObject	Remove a placement and release eligible unused resources.

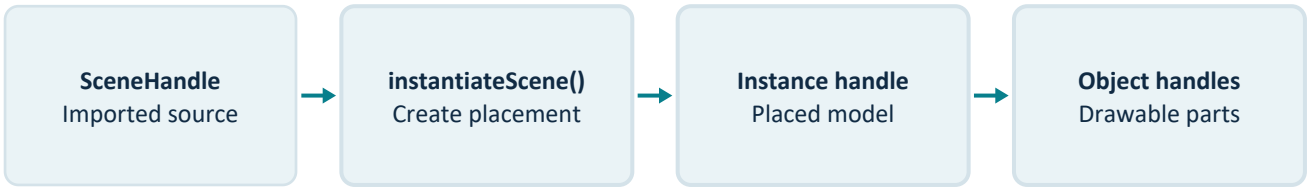
Object queries and changes can fail for stale/wrong handles. Editing mesh positions does not replace indices or recalculate normals. setObjectTransform() sets one object's world placement and moves nothing else. To move an imported part together with the parts below it, use a keep_hierarchy scene instance (page 16).

A repeated texture is one surface
addTexturedPlane(..., Vec2{10,10}, ...) stretches UV coordinates over ten repeats in each direction. The simple renderer uses repeat addressing outside 0..1. You get one plane with a repeated texture; you do not need to tile cuboid geometry. The facade does not expose a general sampler or custom shader editor.

Source: [S37], [S35], [S28], [S18], [S55] | Maps: 76-78

RenderSystem: imported scenes and lifetime

Keep source identity, placement identity and object identity distinct.




```
auto source = assets.loadScene(model_path);
if (!source) { return std::unexpected(source.error()); }
auto instance = render.instantiateScene(*source);
if (!instance) { return std::unexpected(instance.error()); }
auto objects = render.sceneInstanceObjects(*instance);
```

API	Meaning
instantiateScene(SceneHandle, options)	Creates an instance from imported source records; returns RenderSceneInstanceHandle.
SceneInstantiationOptions	Geometry defaults true; imported cameras, imported lights and keep_hierarchy default false.
sceneInstanceObjects(instance)	Find the RenderObjectHandles belonging to the placement.
objectSourceScene(object)	Returns the owning instance, despite its source-style name.
objectSourceNode(object)	Returns the imported NodeHandle of a source part.
setSceneInstanceNodeTransform(instance, node, local)	keep_hierarchy only: sets the node's local transform; every object below the node follows.
sceneInstanceNodeTransform(instance, node)	keep_hierarchy only: the node's current local transform.
removeSceneInstance(instance)	Removes its objects, node tree and owned lights/cameras.
removeScene(source)	Removes render caches for a source; rejects removal while live instances still use it.
purgeUnusedAssets()	Reclaims eligible unused rendering records; returns a count.

One source can have several placements that share resources. Keep a placement handle for group removal and object handles for single parts. By default an instance is flat: each object receives its composed world transform once. With keep_hierarchy the instance keeps the node tree, so setting a mesh-free node such as a car root or a wheel mount moves every part below it. A flat instance returns Error::invalid_argument.

clearScene() resets the rendering scene; loadSampleScene() supplies a built-in sample. Both are scene-wide operations. Removing render state does not erase the imported AssetSystem catalog.

> **A CPU removal is not instant GPU destruction**
> The simple implementation delays release of GPU resources while older submitted frames may still use them. Your game calls removal once and stops using the old handles; the renderer manages that completion-dependent cleanup.

Source: [S37], [S38], [S09], [S28], [S14], [S55], [S56] | Maps: 76-78

RenderSystem: views, lights and observation

These APIs describe a picture and let you inspect what was drawn.

API family	Contract
setCamera(Camera)	Set the default view used by windows without an override.
setCamera(window,Camera) / clearCamera(window)	Set a per-window override / restore the default. After init, invalid, closed or opted-out windows fail.
setDirectionalLight / addDirectionalLight	Replace that light type / append a light. Accept descriptor overloads.
setPointLight / addPointLight	Point position, color, intensity, range and optional ambient; descriptor overloads.
setSpotLight / addSpotLight	Position, direction, color, intensity, range, cone and optional ambient.
clearLights()	Remove all lights while keeping objects, instances and cameras.
Scene statistics	sceneMesh/Material/Texture/Camera/Instance/Vertex/IndexCount and directional/point/spot light counts; hasSceneCamera/DirectionalLight/PointLight/SpotLight.
Rendering statistics	renderedFrameCount(), renderingFramesPerSecond(), lastRenderedWindowCount().
PNG capture	captureFrameToPng(path) or captureFrameToPng(window,path); check expected success.
Shadow inspection	setShadowDepthReadback(bool), then shadowDepthSamples() returns diagnostic copies.

The current simple engine keeps separate light lists. A set call replaces its own type, while add appends up to the supported cap (ten of each type); excess additions are ignored. Its ambient lighting currently uses the x component as a scalar, so equal RGB ambient values are the predictable choice.

Camera owns position, forward, FovY and ClipPlanes. Default near/far distances are 0.1 and 100. Camera::lookAt() derives forward from target minus position. Its up argument is retained for compatibility but ignored; this renderer uses world up. Every window computes its own aspect ratio.

> **Records versus active view**
> Imported cameras added by SceneInstantiationOptions populate scene camera records; explicitly call setCamera() to choose the active view. Likewise, a facade step is not proof that every window drew: minimized or unusable targets can skip drawing. Use the rendering counters and capture results for observation.

Source: [S37], [S29], [S47], [S14], [S26] | Maps: 76-78

GuiSystem and post-processing callbacks

The facade hands control back to your code at defined rendering stages.

GuiSystem has one public drawing hook

GuiSystem::draw(std::function<void()>) stores a callback. The backend invokes it while preparing the GUI for rendering. Calling draw() again replaces the previous callback, so draw all panels from one callback or from functions that callback calls.

```
// Include the ImGui header in the application before imports.
// Register from a long-lived game object, for example in init.
world.template get<vve::GuiSystem>().draw([this] {
    ImGui::Begin("Game");
    ImGui::Text("Score: %d", score);
    ImGui::End();
});
```

The callback receives no World argument; capture the state it needs. Captured references and this must stay alive for every later draw. Registering a function does not copy the objects it refers to. Keep game state in a system owned by the engine, rather than in a helper local that disappears.

RenderSystem supplies effect setup

setPostProcessSetup() takes a function accepting vvppl::PostProcessing&. The simple renderer calls it when creating a target's effect chain. Configure effects there; keep references to settings only while their engine-owned chain remains alive. Resize preserves the retained chain/settings. Multiple targets have separate chains, so retain per-target controls deliberately.

Your callback	Your responsibility
GUI drawing	Call Dear ImGui to create panels; keep captured state alive.
Effect setup	Use the external VVPPL effect API; configure each created chain.
Game update	Change rules/state and publish values before rendering.

Backend boundaries
GUI rendering is managed by the simple backend and currently attached to its first rendering target. Setup failures can disable GUI; a caught std::exception from its callback is reported as a platform error on a later step. This callback path is not a general exception handler for user-system hooks.

Source: [S41], [S23], [S37], [S13], [S43] | Maps: 76-78

BasicECS and ECS: your component store

Entities identify gameplay records; component types are ordinary C++ types.

ECS is the alias for BasicECS<>, using DefaultECSTraits. The implementation owns the actual store and World returns it directly; it is not a forwarding subsystem wrapper. Components are grouped by their C++ type in separate pools.

Call	Meaning
create() / exists(entity)	Create a typed Entity / test whether it is alive.
erase(entity)	Remove the entity and all its stored components.
add<T>(entity,value)	Add a new component; duplicate type on that entity is an error.
get<T>(entity)	Return an expected copy; missing entity/component is an error.
tryGet<T>(entity)	Return expected<optional<T>,Error>; a valid entity may lack the component.
put<T>(entity,value)	Insert or replace the component value.
has<T>(entity)	Return expected<bool,Error> for component membership.
remove<T>(entity)	Remove that component; a missing component on a valid entity is harmless.
view<T...>()	Return a vector of alive entities with all requested component types.

```
struct Health { int value = 100; };
auto& ecs = world.get<vve::ECS>();
auto entity = ecs.create();
auto added = ecs.add(entity, Health{});
if (!added) { return std::unexpected(added.error()); }
auto health = ecs.get<Health>(entity);
if (!health) { return std::unexpected(health.error()); }
health->value -= 10;
auto saved = ecs.put(entity, *health); // Write the copy back.
```

view() is a snapshot of matching entity identities, not a persistent query object. Component values returned by get/tryGet are copies. DefaultECSTraits prepares a choice of storage strategy, but slot-map storage is currently disabled/unimplemented; do not infer stable references or generation-based reuse from that vocabulary.

> **ECS does not automatically render a component**
> Store a RenderObjectHandle in a component and write a system that publishes your Transform to RenderSystem. Removing an Entity does not remove that object automatically, and removing an object does not erase the Entity. Your game owns the bridge between the two identities.

Source: [S39], [S30], [S51], [S37] | Maps: 76-78

Typed handles and small value helpers

An ID says which thing you mean; it does not keep that thing alive.

Public handle	Identifies
Entity	A live gameplay entity in ECS.
WindowHandle	A managed runtime window.
SceneHandle / NodeHandle	Imported source scene / source node.
MeshHandle / MaterialHandle	Imported source geometry / material record.
LightHandle / CameraHandle	Imported light / camera record.
RenderSceneInstanceHandle	A placement of imported scene content.
RenderObjectHandle	One drawable object placement.

Each alias uses TypedHandle<Tag>. Tags are empty structs that distinguish C++ types. A WindowHandle cannot accidentally be supplied where a MeshHandle is required, even though both store a 64-bit value. Zero is invalid.

Utility functions are not resource creation APIs

valid() checks nonzero bits, not whether an object is still in a service. isCounter(), isSlotMapIndex(), generation(), id() and slotIndex() inspect the prepared representation. HandleHash hashes the payload for maps. Comparison operators compare values.

makeCounterHandle() creates a process-local typed counter identity. makeHandleForTest() and makeSlotMapHandleForTest() create deterministic representations for tests. Making an ID does not create a window, model or entity. Obtain resource IDs from their service calls and use service queries to validate their current existence.

Shared sequence and math vocabulary

Vector<T> is std::vector<T>, so normal growth and reference/iterator invalidation rules apply. Scalar, Vec2, Vec3, Vec4, Quat and Mat4 are public GLM-backed math types. CMake VVE_MATH_USE_DOUBLE is off by default (float); enabling it selects double aliases. Use facade names in game interfaces.

> **Three different kinds of handles**
> Public typed IDs identify engine records. C++ references reach stored objects. Vulkan handles name GPU/platform resources inside the implementation. Passing a RenderObjectHandle does not expose a VkBuffer, and keeping the ID after removal does not preserve any resource.

Source: [S50], [S51], [S52], [S49], [S29] | Maps: 76-78

Shared data types: a complete reading map

Small descriptors give names and units to values passed through the APIs.

Types	Meaning and use
Position / Direction / Scale / Rotation	Vec3 location / vector direction / per-axis factors / quaternion. Defaults give zero location, forward direction, unit scale and identity rotation.
Transform	translation, rotation, scale together; interpreted as local or world placement by the receiving API.
LinearColor / LightIntensity / LightRange	Linear RGB / intensity scalar / influence distance.
SpotConeAngle / LightKind	Cone in radians / imported light category.
LightDescriptor	Imported light data carrying kind, position, direction, color, intensity, range, cone and ambient.
DirectionalLight / PointLight / SpotLight	Convenient light request structs for the corresponding render APIs.
FovY / ClipPlanes / Camera	Vertical angle in radians / near-far distances / active view description.
CameraDescriptor	Imported camera data, including aspect/up and separate near/far scalar fields.
PixelExtent / ObjectName / RendererId	Width-height dimensions / owning name string / renderer selection string.
WindowDesc / WindowInfo / WindowFrameData	Startup description / copied runtime state / per-frame vector of copied states.
ApplicationName / MaxFrames / FrameCount	Name option / configured step limit / strong count or index.
DeltaTime / FrameContext / FrameStatus	Seconds / frame index and elapsed time / running or stopped result.
VertexCount / IndexCount	Strong source-geometry count wrappers.
PlaneDescriptor / CuboidDescriptor	Local geometry dimensions, surface color and placement.
TexturedCuboidDescriptor	Local geometry dimensions, base-color path and placement.
SceneInstantiationOptions	Choose geometry, imported lights and camera records, and keep_hierarchy for movable node trees.
Bounds	Axis-aligned minimum/maximum plus valid flag.
RenderShadowDepthSample	CPU/GPU shadow diagnostic values, location and readback-validity flag.

Set the fields relevant to a request, then pass the value to a service. Altering a local Camera or Transform after a setter call does not automatically change the stored copy: call the setter again. FovY and cone angles are radians; DeltaTime is seconds.

identityQuat() uses quaternion constructor order w,x,y,z and avoids confusing identity with a zero quaternion. identityMat4(), zeroVec3(), oneVec3(), zero() and one() are available helpers. vve::math contains vector, quaternion, matrix and projection operations.

Source: [S29], [S49] | Maps: 76-78

Errors, files and the public boundary

Understand a call result before stepping into the implementation.

Fallible APIs return expected values

```
auto result = render.setObjectTransform(object, transform);
if (!result) {
    std::cerr << vve::errorName(result.error()) << '\n';
    return std::unexpected(result.error());
}
// Use *result for an expected<T,Error> with a value.
```

Error is the shared enum; errorName() turns it into a readable name. An expected result has either the requested value/success or an Error. optional means something may be absent without implying a failure, such as findWindow() or the parent of a root node. Some queries combine both: a failed handle lookup and a valid missing parent are different cases.

What goes in .ixx and .cpp

The .ixx files declare exported modules, classes, shared types and public signatures. Templates and small inline definitions live there because importing code needs their definitions. Engine, EngineBuilder, World and ECS therefore contain their template implementations in module interface files.

Window.cpp, Assets.cpp, RenderSystem.cpp and Gui.cpp contain out-of-line facade method definitions. Most wrapper methods forward to impl_. The selected simple engine has its own .ixx/.cpp files for actual storage, importing, platform work and Vulkan rendering. The suffix alone does not tell you whether code is public or internal; its module, namespace and exported contract do.

makeEngineImpl() is already inline in Engine.ixx. There is no separate facade Engine.cpp needed for that factory. Moving one factory does not remove the need for other .cpp implementation files, whose non-template definitions still have work to do.

When reading source	Start here
Creation and hooks	src/Engine.ixx, src/World.ixx, selected implementation adapter.
Public request contract	src/Window.ixx, Assets.ixx, RenderSystem.ixx, Gui.ixx and Types.ixx.
Gameplay storage	src/ECSContainer.ixx and Entity.ixx.
How it is executed	The simple implementation walkthrough in Part II.

Use the contract to write a game
API signatures tell you what you may request. The implementation explains timing, resource sharing and supported behavior. Part III turns those contracts into a starter game and programming recipes. The listings in this book are source-checked explanations; newly written snippets have not been compiled as separate executables.

Source: [S01], [S32], [S39], [S42], [S54] | Maps: 76-78

Part II: the simple engine

Step through initialization, assets, resource storage, Vulkan commands and frame execution.

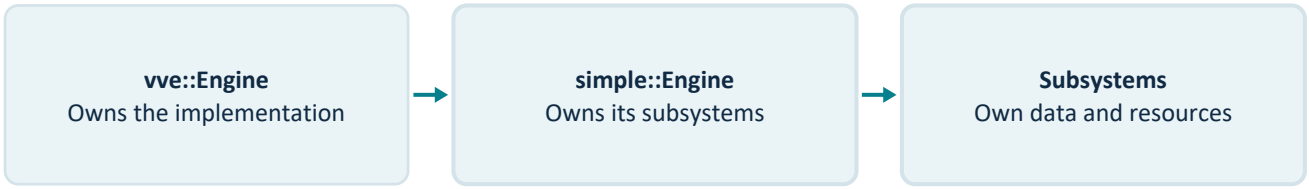
Page	Chapter
24	From the facade into the simple engine
25	The main classes
26	Choosing startup settings
27	Creating windows and input
28	What one engine step does
29	Loading a model file
30	Where imported data is stored
31	Turning assets into visible objects
32	Textures, pixels and repeating UVs
33	Starting Vulkan
34	Resources for each window
35	Memory and allocation
36	Uploading meshes and textures
37	Telling shaders where data lives
38	Acquiring and drawing a frame
39	Recording the drawing passes
40	What shaders and shadows do
41	Keeping CPU and GPU work safe
42	Changing and removing resources
43	Multiple windows and resizing
44	GUI, effects and frame capture
45	Stopping and releasing resources
46	Worked example: startup
47	Worked example: one object, one frame
48	Class reference: engine and scene
49	Class reference: GPU resources
50	Plain-language glossary

The source maps follow the game guide. Each chapter footer lists the supporting source labels. Start at the facade-to-implementation bridge and read through the frame before consulting the class reference tables.

From the facade into the simple engine

Follow the public request through its stored reference to the concrete implementation.

Your game imports **VVEngine** and uses classes in **vve**. The simple implementation lives in **vve::simple**. The facade forwards requests to those implementation objects.



What ownership means

An owner keeps an object alive and is responsible for releasing it. The facade Engine owns a simple::Engine through a unique_ptr. The simple engine contains its WindowSystem, AssetSystem, RenderSystem, GuiSystem and ECS objects.

The public subsystem wrappers hold references to those implementation objects. A reference is a way to reach an object that already exists. Calling the public renderer therefore reaches the same renderer object on every frame.

```
auto world = engine.world();
auto& render = world.get<vve::RenderSystem>();
auto& assets = world.get<vve::AssetSystem>();
```

World is an access point

World contains references to the subsystem wrappers and your registered game systems. It gives you access by type with get<T>(). It does not create another world of models or GPU resources. Its references must remain within the engine lifetime.

CMake selects an adapter using VVE_ENGINE_IMPLEMENTATION_NAMESPACE. The current adapter defines EngineImpl as simple::Engine and the equivalent aliases for the other subsystems. Currently only simple is available.

> **Type and object**
> The alias chooses the class. makeEngineImpl() creates an actual object of that class. The stored pointer or reference chooses the object on which a method runs.

Source: [S01], [S02], [S03], [S32] | Maps: 76-78

The main classes

Each part has a particular job and keeps the information needed for that job.

Class or object	Its everyday job
simple::Engine	Coordinates the platform and rendering parts. Keeps startup settings and frame timing.
WindowSystem / Window / InputState	Own windows, translate SDL events, and remember held keys and mouse changes.
AssetSystem / Catalog	Read files and keep their imported content in CPU memory.
RenderSystem / RenderScene	Create and manage the objects that should be drawn.
ForwardRenderer	Create Vulkan resources, upload changes, record drawing commands and present images.
WindowTarget	Keep one window's swapchain, image attachments, frame slots and camera override.
GuiSystem	Build and draw the immediate-mode user interface.
ECS	Keep entities and their components. This is shared code from the common layer.

Objects remember state

A class describes what an object can contain and do. An object contains the current values. WindowSystem remembers its windows; Catalog remembers imported meshes; RenderScene remembers object transforms; ForwardRenderer remembers GPU buffers.

Your game systems are separate objects

Engine<MovementSystem, GameSystem> stores your system instances in the facade. Their init and update hooks are called there. The simple engine supplies input, timing and drawing. The ECS does not automatically turn every component into a rendered object; game code connects its data to RenderSystem.

Source: [S03], [S04], [S05], [S08], [S12], [S23], [S30], [S01] | Maps: 76-78

Choosing startup settings

Construction collects settings and builds the objects that will later be initialized.

```
auto engine = vve::EngineBuilder<>{}
    .applicationName("My game")
    .addWindow(vve::WindowSetup{}
        .id("main")
        .title("My game window")
        .extent(vve::PixelExtent{.width=1280, .height=720})
        .renderer(vve::RendererId{.value="forward"}))
    .build();
```

build() calls the facade Engine constructor. It gathers typed startup options, calls makeEngineImpl(), connects public wrappers to the implementation subsystems, and stores your user systems. It does not open an SDL window yet.

Engine option	Default and meaning
ApplicationName	"simple". A custom name replaces window titles that still have the default value.
MaxFrames	0 means unlimited. A positive value caps engine steps, not frames per second.
WindowSetups	An omitted or empty list produces one default window.
UserSystems	Default-construct the system types named in Engine<...>, or supply objects with makeUserSystems().

Window choices

WindowSetup offers id, title, extent, position, renderer, resizable and visible. Defaults are main, VVE simple, 960 x 540, platform placement, forward rendering, resizable and visible. Startup size uses window coordinates; drawable pixel size can be larger on a high-DPI screen.

The renderer IDs "" and "forward" draw; "none" opts a window out. A hidden window still renders. The builder can add several windows. Its windows() call replaces the list, while addWindow() appends.

> **Supplying game systems**
> A system without a default constructor needs a supplied UserSystems bundle. The current builder copies systems during build(); direct Engine construction can move a temporary bundle and supports move-only systems.

Source: [S01], [S03], [S29] | Maps: 76-78

Creating windows and input

init() creates operating-system resources; input is refreshed on every step.

The window initialization path

The public Engine::init() first calls simple::Engine::init(). The simple engine checks renderer IDs and asks WindowSystem::init() to create the configured windows. The facade then initializes your game systems.

WindowSystem first checks every descriptor: IDs must be nonempty and unique, and width and height must be nonzero. It starts SDL video and creates the SDL windows. Rendering windows use Vulkan-capable SDL windows.

New windows are kept as pending objects until the whole operation succeeds. If a creation fails, pending windows are destroyed and that SDL video reference is released. On success, the windows are committed together. The Engine-level init() is safe to repeat after success.

Class	What it keeps
WindowSystem	The owned window collection, SDL lifetime, input object and event lookup maps.
Window	An SDL_Window pointer and cached WindowInfo: ID, title, pixel size and state.
InputState	Held keys, this-frame presses/releases, mouse positions and per-window deltas.

How events become useful input

At the start of polling, InputState clears changes that only belong to one frame. WindowSystem processes SDL events: key presses and releases, mouse movement, resizing, focus, minimizing and close requests. Held keys stay held until a release arrives.

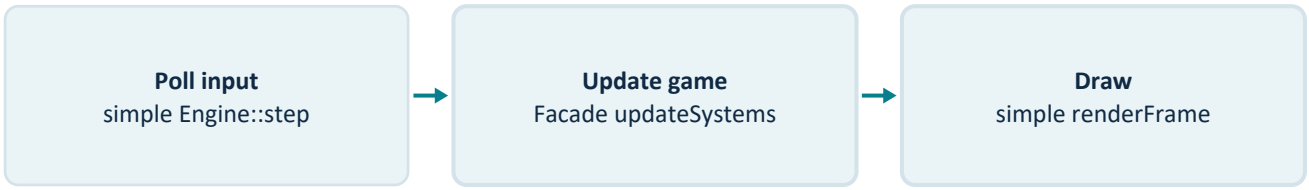
Events also reach the GUI. If the GUI claims a key-down or mouse event, the game input avoids using it. Key releases still reach the game so keys cannot remain stuck down. Cached window information is updated as events arrive.

> **The next stage is still separate**
> Successful window initialization does not mean the Vulkan device and scene resources exist. The first drawable rendering attempt creates them.

Source: [S03], [S04], [S23] | Maps: 76-78

What one engine step does

The facade puts game updates between platform polling and rendering.



The exact order

1. The facade calls simple::Engine::step(). It checks initialization, consumes any pending GUI callback error, and polls WindowSystem.

2. It stores FrameContext: the current zero-based frame index and elapsed time. The first delta defaults to 1/60 second; later deltas use a steady clock. It advances the step counter and checks closing or the frame cap.

3. The facade gets FrameContext and calls your registered systems in template-argument order. It supports update(world), update(world, frame), or update(world, frame, windowFrame). The most detailed available form is selected.

4. The facade calls simple::Engine::renderFrame(). It decides whether windows can draw, starts rendering lazily if needed, and delegates the actual drawing to RenderSystem.

```
const auto status = impl_->step();
// Check the result.
const auto frame = impl_->frameContext();
// updateSystems(frame), with error checking.
// impl_->renderFrame(), with error checking.
return *status;
```

Stopping and failure

A stopped status can still reach the final updates and drawing before returning. Closing any window stops the normal run loop. run() calls init() and repeats step(). A manual step() requires prior initialization. If polling fails, updates do not run. If a game update fails, later systems and drawing are skipped for that step.

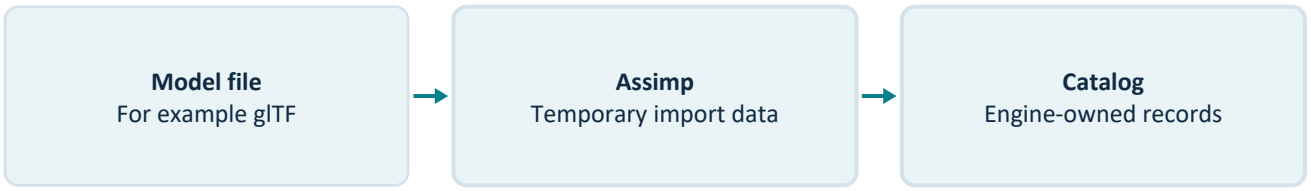
EngineBuilder<> registers no game systems. In the current game example, gameplay runs explicitly in main() before engine.step(), so it reads the input snapshot from the previous step.

Source: [S01], [S03], [S34] | Maps: 76-78

Loading a model file

AssetSystem turns an external file into data the engine owns.

```
auto world = engine.world();
auto& assets = world.get<vve::AssetSystem>();
auto scene = assets.loadScene(modelPath);
if (!scene) { /* handle scene.error() */ }
```



What loadScene() does

It checks the path and creates a temporary Assimp importer. Assimp reads the file and helps convert it: faces become triangles, identical vertices can be joined, normals and tangents are generated where needed, and mesh ordering is improved.

AssetSystem then creates its own records. Materials are imported with colors, roughness, metalness, emission and texture sources. Meshes receive vertices and triangle indices. Lights and cameras receive descriptors. Nodes receive names, local transforms and mesh references.

The importer is temporary. The engine copies or builds the data it needs in its own containers before the importer disappears. Keeping a SceneHandle therefore does not depend on Assimp's temporary memory.

What the returned handle means

SceneHandle is a typed ID used to find the imported scene. It is neither the model bytes nor a GPU address. The scene is now available in CPU memory for queries or later instantiation.

> **Loading is not drawing**
> loadScene() populates the asset catalog. The render system needs instantiateScene() to create draw objects. A later rendering attempt creates or uploads the required GPU resources.

A failed file import reports asset_import_failed. An empty source path reports invalid_argument. Repeating loadScene() is another import call; sharing render resources is handled later using the imported handles.

Source: [S05], [S07], [S33] | Maps: 76-78

Where imported data is stored

Catalog is the collection of engine-owned records; the tree describes their arrangement.

Record	Stored information
Catalog / Table<T>	Maps typed handles to AssetScene, Node, AssetMesh, Material, Light and CameraAsset records.
AssetScene	Scene identity, name, node tree and lists of mesh, material, light and camera handles.
Node	Name, local transform and attached mesh handles.
AssetMesh	Name, material handle, bounds, shared immutable vertices and triangle indices.
Material	Base color, material factors and typed texture sources.
Light / CameraAsset	Imported lighting or camera descriptors.
Graph<T> / Tree<T>	Node membership, parent/child links and creation order.

An example hierarchy

```
Scene: house
  Root node
    Wall node   -> mesh A -> material stone
    Roof node   -> mesh B -> material tiles
    Door node   -> mesh C -> material wood
```

Nodes carry local transforms relative to their parents. When instantiating, the renderer computes a world matrix as parent world matrix times child local matrix, so the door keeps its place relative to the house. A keep_hierarchy instance also keeps this tree: setting the root node later moves the door with it (page 31).

Why vertices are shared

An imported AssetMesh owns vertices through a shared_ptr to an immutable vector of RenderVertex. Render meshes can share that same CPU payload. The bridge copies indices once. If an object's vertices are edited, the engine detaches the affected shared data before writing, protecting the original imported mesh.

Each vertex contains position, surface normal, UV texture coordinates and tangent information. Indices describe which vertices make each triangle. Public asset queries return owning values; internal borrowed pointers and spans have shorter lifetimes.

Source: [S05], [S06], [S07], [S09] | Maps: 76-78

Turning assets into visible objects

RenderSystem builds the CPU scene that the renderer will draw.

```
auto& render = world.get<vve::RenderSystem>();
auto instance = render.instantiateScene(*scene);
if (!instance) { /* handle instance.error() */ }
```

RenderSystem uses ImportedAssetReadAccess: a bundle of functions supplied by the engine. These functions ask AssetSystem for nodes, transforms, meshes, materials, lights and cameras without handing the renderer the whole Catalog.

instantiateScene() walks the node hierarchy and composes world transforms. For each usable mesh it acquires a render mesh and material, creates a RenderInstance, and registers a public RenderObjectHandle. It also remembers which source scene and node produced that object.

CPU render record	Purpose
RenderMesh	Shared vertices plus its own index array.
RenderMaterial	Surface factors and texture-table indices.
RenderInstance	Mesh and material handles, local/world transforms, visibility, shadow and unlit flags.
RenderScene	Containers and lookup maps for meshes, materials, instances and texture entries.
Scene	The renderer's lighting records; geometry lives in RenderScene.

Several placed objects can refer to the same mesh and material. Import caches reuse resources by source handles. Moving one object changes its transform; it does not require another copy of the original model.

Scene instance options

Geometry is enabled by default; imported lights, imported cameras and keep_hierarchy are opt-in. apply_cameras records imported camera ownership/counts but does not select the active view: call setCamera(). Imported texture failures fall back to untextured material slots instead of failing the instance.

Instances that keep their hierarchy

With keep_hierarchy, instantiateScene() also copies the node tree into scene_hierarchies_. Each HierarchyNode stores its parent, children, current local transform and the objects made from its meshes.

setSceneInstanceNodeTransform() saves the new local transform, multiplies the unchanged ancestors once, then walks the subtree: every object below gets world = parent world * local as its exact world matrix. removeObject() unlinks an object; removing the instance drops its tree.

Source: [S08], [S09], [S10], [S11], [S29], [S56] | Maps: 76-78

Textures, pixels and repeating UVs

The material points to a texture entry; decoding produces pixels; the GPU later receives an image.

MaterialTextureSource remembers a texture's role and source. The source can be an external image path or embedded image bytes from the model. RenderScene::acquireTexture() finds or creates the corresponding render texture entry.

Entries are shared by canonical source identity and color-space use. Base color and emissive maps use sRGB; normal, roughness, metalness and other data maps use linear sampling. The same file used in both roles can require two entries.

stb_image decodes images into RGBA8 bytes. Embedded raw pixels can be copied directly. Rows are flipped to match the imported/procedural UV convention. RenderTexture keeps these pixels, dimensions, source, color-space flag and a generation number.

Term	Plain meaning
UV coordinates	Coordinates on the surface telling the shader where to sample the texture.
Sampler	Rules for filtering and for coordinates outside the normal 0 to 1 range.
Mip levels	Progressively smaller versions of the texture for distant or small surfaces.
Generation	An identity used to check whether the GPU holds the current texture content.

Why your ground texture repeats

addTexturedPlane() scales the plane's UVs. The shared material sampler uses REPEAT addressing. A coordinate like 4.25 samples the same horizontal location as 0.25, after four repetitions. The geometry remains one plane; the texture supplies the tiling.

What stays after upload

When the matching texture generation is on the GPU, RenderSystem releases its temporary decoded CPU pixels. It retains the file path or embedded source. A later GPU reinitialization can decode again. Decode failures are cached until clearScene(); the current table has 64 texture slots.

Source: [S07], [S10], [S11], [S13], [S18], [S28] | Maps: 76-78

Starting Vulkan

GPU startup waits until the engine has a window that can draw.

simple::Engine::renderFrame() checks the windows. Closed, minimized, zero-size and renderer-none windows do not draw. If at least one window is drawable and rendering is not initialized, RenderSystem::initialize() starts the GPU path.

The first rendered window starts shared resources

Step	What is acquired or created
1. VulkanInstance	A connection to the Vulkan loader and enabled instance capabilities.
2. VulkanSurface	A connection between an SDL window and Vulkan presentation.
3. VulkanPhysicalDevice	A suitable physical GPU and graphics/presentation queue families.
4. VulkanDevice	The logical device and queues used to issue work.
5. VulkanAllocator	The VMA allocator used for buffers and images.
6. Drawing setup	Descriptor layout, pipeline layout, embedded shader modules, forward/shadow pipelines and command pool.
7. Shared scene resources	Shadow arrays, fallback white texture, scene textures, GPU meshes and packed materials.
8. WindowTarget resources	Swapchain, views, depth/HDR images, frame slots, synchronization and bindings.

The selected GPU must support the code's requirements, including Vulkan 1.3 device features, swapchain presentation, dynamic rendering and dynamic texture-array indexing. The instance currently requests Vulkan 1.4. Optional anisotropic filtering is enabled when supported.

ForwardRenderer owns the shared device, allocator, pipelines, textures, mesh buffers and shadow maps. Additional windows receive their own WindowTarget resources while sharing these device-wide objects.

> **Initialization has real failure points**
> Window init can succeed and GPU startup can fail later. Most setup stages stop at the first Vulkan error and clean up the created rendering resources. Renderer errors generally reach the facade as platform_error.

Source: [S03], [S08], [S12], [S13], [S16] | Maps: 76-78

Resources for each window

WindowTarget contains what one window needs to render and show its own image.

A surface identifies where Vulkan can present. A swapchain is a set of images used to show successive pictures in that surface. Vulkan provides the swapchain images; the engine keeps borrowed handles to them and creates views for drawing.

WindowTarget resource	What it is for
VulkanSurface / VulkanSwapchain	Connect the SDL window to a sequence of presentable images.
VulkanImageViews	Allow an image to be used as a particular attachment or view.
depthImage	Remembers which visible surface is nearest at each pixel.
hdrImage	Offscreen RGBA16F image where the lit scene is drawn before effects and presentation.
VulkanCommandBuffers	Store a recorded list of GPU instructions for each frame slot.
VulkanFrameSync	Fences and semaphores that coordinate safe reuse and presentation.
Uniform/material buffers and descriptor sets	Give the shaders this window's frame data and resource bindings.
Camera / PostProcessing	Optional window-specific view and effect chain.

Two frame slots

Each window has two frames in flight. A slot contains commands and frame data that can be reused after its fence completes. This lets the CPU prepare work while earlier GPU work is finishing.

Two frame slots do not mean two swapchain images. The swapchain image count is chosen using surface capabilities. The frame slot index and acquired image index serve different purposes.

Shared attachments still need ordering

The depth and HDR attachments are per-window, while the frame slots hold separate commands and data. Image barriers and ordered submissions protect shared attachment use. Shadow arrays are shared between windows, and their access is also ordered.

Source: [S12], [S13], [S20], [S21] | Maps: 76-78

Memory and allocation

C++ containers hold CPU data; Vulkan buffers and images hold resources the GPU can use.

VulkanAllocator wraps VMA, the Vulkan Memory Allocator. It finds suitable memory and manages allocations for buffers and images. It does not decide what a model means; it supplies storage for bytes.

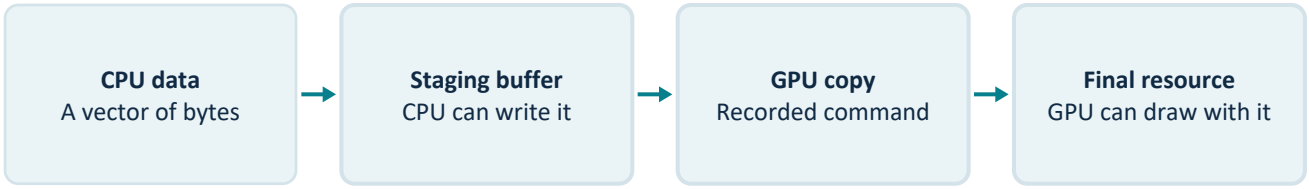
Memory intent	Used for	How data reaches it
device_local	Static vertex/index buffers and image storage	GPU-side copies from staging resources.
upload	Staging buffers, frame uniforms, material tables and edited vertices	CPU writes to a persistent mapped pointer.
readback	Results the CPU needs to inspect	GPU copies into a buffer; CPU reads after completion.

Device-local is a Vulkan memory property, not a guarantee that every computer has separate physical VRAM. On integrated GPUs, CPU and GPU memory may share physical hardware. The engine still follows the declared access rules.

The two basic resource owners

VulkanBuffer owns a byte buffer and its VMA allocation. Host-visible buffers also keep a mapped pointer. upload() copies bytes into that mapping. This engine asks for coherent host memory, so these writes do not need a separate explicit flush.

VulkanImage owns an image allocation, an image view, and optionally layer views. Textures, depth attachments, HDR attachments and shadow maps build on this image machinery.



Mapped memory is only a way to access bytes. It does not remove the need to wait before overwriting data still used by submitted GPU work. Allocation, writing and synchronization are separate responsibilities.

Source: [S17], [S18], [S20] | Maps: 76-78

Uploading meshes and textures

An upload is a transfer into GPU resources, performed only when resources are new or changed.

Uploading one mesh

VulkanMesh::create() allocates temporary upload buffers, copies CPU vertices and indices into them, and creates device-local vertex and index buffers. A temporary command buffer records both vkCmdCopyBuffer operations and a barrier for later vertex/index reads.

The upload is submitted to the graphics queue and waits for its fence. Once the copy is complete, temporary staging buffers can be destroyed. VulkanMesh retains the final buffers, index count and a cached bounding box.

Uploading textures

TextureImage builds a Vulkan image from RGBA8 pixels. VulkanUploadBatch retains staging buffers while it records copies and generates smaller mip levels with blits, where supported. Images transition from transfer use to shader-read use.

uploadSceneTextures() batches all changed texture images into one submission and fence wait. It updates the recorded generations only after a successful upload. Unused texture bindings use a shared opaque-white fallback.

Reusing and editing

syncSceneResources() checks the current CPU render scene. It uploads missing live meshes, rebuilds material data when needed and uploads changed textures. Existing resources with unchanged identities are reused.

When mesh positions change, static vertices are retired and the engine uses per-window, per-frame-slot mapped vertex copies. Only the slot whose fence has completed receives its pending update. Index topology and allocation size must still match the supported edit.

> **Uploads are not the whole frame**
> Upload helpers wait for their own temporary fence. Normal frames use two reusable slots and their own fences. Routine scene changes do not require a global device-idle wait.

Moving an object normally only changes its transform. It does not upload an entirely new mesh. Sharing a mesh across several instances also avoids uploading it several times.

Source: [S13], [S17], [S18], [S28], [S35] | Maps: 76-78

Telling shaders where data lives

Descriptors identify resources; buffers contain their data; push constants supply small draw-specific values.

A GPU program cannot directly follow a C++ std::vector or map. The engine gives it Vulkan resources and bindings that match the layout declared in the shaders.

Mechanism	Contents	How it is used
FrameUniforms / VulkanUniformBuffers	View and projection matrices, camera, lights and shadow matrices	A mapped buffer for each window frame slot.
GpuMaterial / material buffer	Color and material factors, texture-table indices	A dense shader-readable table; public material handles map to table slots.
Descriptor set	References to frame data, texture array, shadow arrays and material buffer	Bound before drawing. Points to resources; does not contain the texture pixels.
ObjectPushConstants	Model matrix, material slot, shadow-matrix index and unlit flag	Small values recorded with each object draw.

The classes that build the binding contract

VulkanDescriptorSetLayout describes the shader bindings. VulkanDescriptorPool supplies space to allocate sets. VulkanDescriptorSets owns allocated sets and writes resource references into them. VulkanPipelineLayout combines those bindings with the push-constant layout.

VulkanShaderModule holds a compiled GPU program. VulkanGraphicsPipeline combines shader stages and drawing settings, such as vertex format, depth testing and target formats. VulkanVertexInputDescription explains how bytes in RenderVertex become shader inputs.

```
vkCmdBindPipeline(...);
vkCmdBindDescriptorSets(...);
vkCmdBindVertexBuffers(...);
vkCmdBindIndexBuffer(...);
vkCmdPushConstants(...);
vkCmdDrawIndexed(...);
```

These calls record instructions into a command buffer. They do not execute the drawing immediately. The GPU runs the recorded work after a queue submission.

Source: [S18], [S19], [S24], [S14] | Maps: 76-78

Acquiring and drawing a frame

drawFrame() prepares a safe slot, obtains an image, submits drawing and requests presentation.

RenderSystem calls ForwardRenderer::drawFrame() for each renderable WindowTarget. It first checks the actual drawable size and rebuilds size-dependent resources if needed.

Order	Action and reason
1. Wait for the slot fence	Make sure the GPU has finished using the slot's commands and mapped data.
2. Synchronize scene resources	Reclaim safe retired objects, upload scene changes, update this slot's bindings and edited vertices.
3. Prepare this window's frame	Choose its camera, compute shadow data, write frame uniforms and prepare GUI draw data.
4. Acquire an image	vkAcquireNextImageKHR returns the swapchain image index that this frame may use.
5. Record commands	recordCommandBuffer() records shadows, scene color, image effects/copy and GUI.
6. Submit the commands	vkQueueSubmit sends work to the GPU, waits on image availability and signals completion sync.
7. Present the image	vkQueuePresentKHR waits for render-finished and requests that the image be shown.

Acquiring is not allocating

The swapchain images already exist. Acquisition gives this frame permission to use one of them. Its image index is independent of the frame slot being reused.

Scene and frame preparation deliberately happen before acquiring the image. If preparation fails, the engine has not yet acquired an image or signaled its acquire semaphore.

The code waits up to one second for a busy frame fence and 100 ms for image acquisition. Timeout/not-ready results skip the attempt and retry later. An out-of-date swapchain is recreated. A suboptimal acquisition still yields an image that must be consumed.

The current drawFrame() returns a boolean: true means presentation succeeded or was suboptimal. Many per-frame Vulkan failures are logged and skip that target rather than becoming an Engine error. Texture upload/decode errors have an explicit path back through RenderSystem.

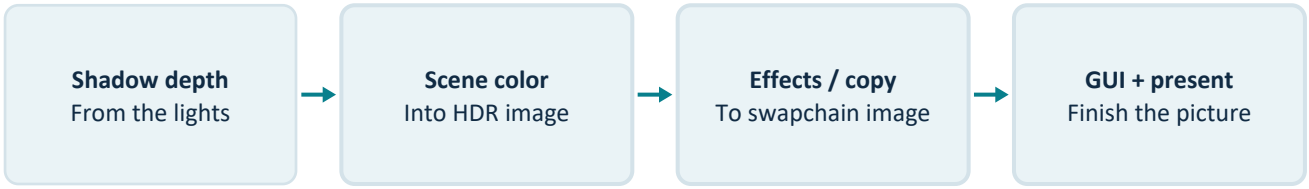
Source: [S08], [S14], [S20], [S21] | Maps: 76-78

Recording the drawing passes

A command buffer is the GPU's instruction list for one frame.

recordCommandBuffer() resets and begins the chosen command buffer. It resolves visible RenderInstances into temporary DrawItem records with GPU buffers, object data and world-space bounding boxes. Items are sorted by mesh to reduce repeated buffer bindings.

Objects outside a pass's viewing volume can be skipped. The engine uses their bounding boxes conservatively: touching or uncertain boxes stay drawable. An object can be invisible or visible but excluded from shadow drawing.



What the passes record

Shadows: the shadow pipeline renders depth from each contributing light's views into layers of depth-image arrays. These layers later tell the lighting shader where surfaces are hidden from the light.

Scene color: the forward pipeline draws into the window's HDR image and depth attachment. For each object it binds the mesh when needed, writes push constants and records vkCmdDrawIndexed().

Effects or copy: the HDR result is passed through the configured vvppl effect chain. Without a chain, a blit copies/converts it to the acquired swapchain image. The code does not automatically add a tone-mapping effect.

GUI: if this is the GUI window and there is draw data, ImGui commands are added on top of the swapchain image. Finally, the image is transitioned to PRESENT_SRC_KHR and command recording ends.

> **Why the image barriers are there**
> An image changes jobs during the frame: depth attachment, sampled shadow map, color attachment, transfer image or presentable image. Barriers order those jobs and make the written contents available to later reads.

Source: [S14], [S19], [S22], [S23] | Maps: 76-78

What shaders and shadows do

Shaders are small programs compiled for the GPU, with a specific role in drawing.

Shader entry	What it does
vertexMain	Reads each mesh vertex. Uses the object model matrix and camera matrices to place it on screen. Passes surface information onward.
shadowVertexMain	Transforms vertices into the selected light view. The shadow pass stores depth rather than scene color.
fragmentMain	Computes a surface sample's color from material factors, texture samples, normals, lights and shadow lookups.

From triangles to colors

The GPU assembles triangles from vertices and indices. It determines which screen samples those triangles cover and interpolates values such as UVs and normals. The fragment shader samples textures and computes the color for those covered samples.

The current shader uses a Blinn-Phong style lighting calculation influenced by roughness and metalness. It supports base color, normal, roughness, metalness, emissive and ambient-occlusion texture roles. This is the current educational lighting model, rather than a claim of a full physically based renderer.

Three light types

A directional light acts like distant sunlight and uses four shadow cascades. A spot light shines in a cone and uses one shadow layer. A point light shines in all directions and uses six depth views, like the faces of a cube.

prepareShadowFrame() packs contributing lights and builds their matrices. ShadowMap owns the depth arrays. The fragment shader compares a surface's distance from the light with the stored depth and samples nearby positions to soften the edge. Objects marked unlit bypass lighting.

When shader programs are acquired

CMake runs slangc at build time. It compiles the Slang entries to SPIR-V arrays embedded in the executable/library. VulkanShaderModule creates modules from those arrays during GPU startup. The renderer does not read Slang source files on each frame.

Source: [S15], [S19], [S22], [S24], [S25] | Maps: 76-78

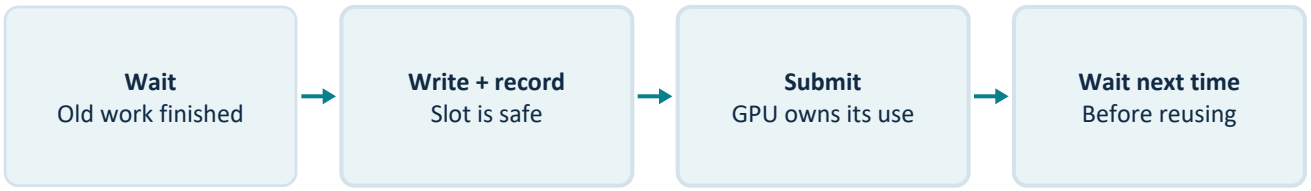
Keeping CPU and GPU work safe

The CPU and GPU work at different speeds. Synchronization prevents premature reuse.

vkQueueSubmit usually lets the CPU continue before GPU execution finishes. Submitted commands may still be reading buffers and images after the C++ function returns. The engine must therefore track which resources are safe to reuse.

Tool	Simple meaning	Used here for
Fence	A completion signal the CPU can wait for	Waiting before reusing a frame slot; waiting for temporary uploads.
Semaphore	A dependency between GPU/presentation operations	Waiting for image availability and waiting for rendering before presentation.
Barrier	Ordering and visibility inside GPU work	Making copies visible; moving images between drawing, sampling, copying and presentation.
Device-idle wait	Wait for outstanding device work	Resize, shutdown, closed-target removal and explicit debugging boundaries.

One normal slot cycle



Acquire semaphores and fences follow the frame slots. Render-finished semaphores follow swapchain image indices because presentation can outlive the slot that submitted the work. These indexing rules are deliberately different.

The in-flight fence starts signaled, allowing the first frame. It is reset only when submission is ready. A reset fence with no submission would leave the next frame waiting for work that never existed.

The normal submission waits for image availability at the transfer stage. Offscreen shadow and HDR work does not need the presentable image yet; the later swapchain transition and copy are ordered after acquisition. Shared attachments are protected by barriers and graphics-queue ordering.

Source: [S14], [S17], [S20] | Maps: 76-78

Changing and removing resources

The CPU can request changes while older GPU work still uses the previous data.

A small transform change

setObjectTransform() changes the CPU RenderInstance transform. The next command recording uses the new model matrix in push constants. Geometry and texture allocations can stay the same.

A mesh or texture change

Dirty flags tell the renderer which parts need synchronization. Texture generations identify changed content. A new GPU resource can be created while an older submission still refers to the old one.

The renderer places old resource owners in a retirement queue with a submission serial. When a completed frame fence covers that serial, collectRetiredResources() releases the old owners. Submission order on the shared graphics queue makes earlier work covered as well.

CPU request	GPU-side follow-up
Move an object	Record a new model matrix; reuse the mesh.
Edit mesh positions	Detach shared CPU geometry if needed; update safe per-slot vertex copies.
Replace/release a texture	Upload a new generation when needed; update each safe descriptor slot; retire old images.
Remove an object or scene instance	Remove draw records; reclaim resources only when no live instances need them and GPU use is finished.
purgeUnusedAssets()	Remove unused CPU render caches; later synchronization removes corresponding unused GPU owners.

Removing a source and removing its placement

A RenderSceneInstanceHandle groups a placed imported scene. Removing it removes its objects and owned imported light/camera records. The asset catalog is separate. Current RenderSystem::removeScene() manages render-side instances/cache entries for that source; the Catalog remains owned by AssetSystem.

> **Why slots have dirty flags**
> One slot may be safe while another is still running. Materials, texture bindings, shadow bindings and dynamic vertices are changed only in the slot whose fence has completed.

Source: [S13], [S28], [S35], [S12] | Maps: 76-78

Multiple windows and resizing

Windows share the device and model resources but keep separate presentation resources.

Shared between windows	Separate for each window
Vulkan device, queues and allocator	Surface, swapchain and image views
Graphics and shadow pipelines	Depth and HDR attachments
GPU meshes, textures and sampler	Frame slots, mapped frame/material buffers and descriptors
Lighting data and shadow arrays	Camera override, dynamic vertex slots and post-processing chain

The first rendered surface participates in selecting the GPU. Each additional surface must be presentable from the existing graphics queue. Targets are drawn in window order. Their camera and aspect ratio are prepared separately.

Cameras

setCamera(camera) sets the default view. setCamera(window, camera) gives one window an override. clearCamera(window) restores its use of the current default. Camera near/far values also control projection and directional shadow coverage.

Resizing

drawFrame() compares the live pixel size with the size requested for the swapchain. recreateSwapchain() waits for device work, replaces image views, the swapchain, depth/HDR attachments and frame synchronization, and resizes the effect chain. The shared device, scene resources and graphics pipelines are retained.

Minimizing and closing

Minimized or zero-size windows skip drawing while retaining their targets and camera/effect settings. If no window is drawable, the engine waits briefly for an event. A hidden window can still draw. A renderer-none window has no render target.

Closed targets are cleaned after a device-idle wait. The normal Engine run loop still treats closing any window as a request to stop. Render counters advance when at least one target presents; engine step counts and presented-frame counts can differ.

Source: [S03], [S08], [S12], [S13], [S21], [S35] | Maps: 76-78

GUI, effects and frame capture

These features use the same device and frame sequence, with their own preparation steps.

GuiSystem

draw(callback) stores your function for building the UI. During first renderer startup, the engine creates an ImGui context, starts its SDL and Vulkan backends and uploads fonts. If setup fails, it cleans the partial setup and continues scene rendering without GUI.

prepareFrame() starts the ImGui frame and runs your callback to build draw data. It happens before image acquisition. record(commandBuffer) later adds the GUI drawing commands during the GUI pass. No UI vertices means the GUI pass is skipped.

Currently only the first rendered target owns the GUI pass and its special color views. The SDL event path still forwards input events from all windows. A caught standard exception in a GUI callback is remembered and reported as platform_error by the next simple engine step.

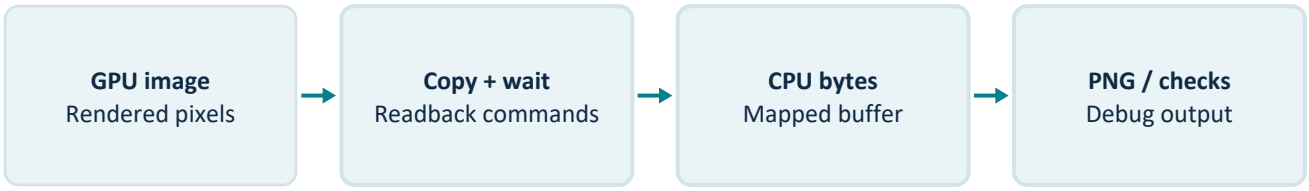
Post-processing

setPostProcessSetup() supplies a function that configures a vvppl::PostProcessing chain for each target when it is created. The chain processes the HDR result before GUI drawing. Its size-dependent resources are resized when that window changes size.

Without a configured chain, the renderer blits the HDR image to the swapchain image. An empty configured chain still passes through the library's apply() call; the engine does not use an empty-chain query.

Reading an image back

captureFrameToPng() uses VulkanReadback to copy GPU image data into CPU-readable memory and writes a PNG. A window-specific overload selects the target. Shadow-depth readback is optional and is useful for checking shadow calculations.



Readback is a deliberate debugging operation and can add waits. It is not the path by which normal frames reach the monitor; normal display uses presentation.

Source: [S03], [S23], [S13], [S14], [S26], [S27] | Maps: 76-78

Stopping and releasing resources

Resources are released in an order that keeps their dependencies alive until they are no longer needed.

The facade owns the simple engine, so destroying the facade eventually destroys the implementation. Destroying a resource owner calls cleanup() or releases its RAII handle. RAII means the lifetime of the C++ object controls the lifetime of the owned resource.

Shutdown order

1. The engine waits for GPU work so no submitted frame still references the resources being released.

2. It shuts down the ImGui Vulkan backend before renderer resources that it depends on are destroyed.

3. ForwardRenderer cleans each WindowTarget: effect chain, descriptor sets/pool, frame and material data, dynamic vertices, sync objects, command buffers, attachments, views, swapchain and surface.

4. It releases device-wide meshes, texture images, retired owners, sampler, readback resources, command pool, pipelines, shader modules, layouts and shadow arrays.

5. It destroys the allocator, logical device and Vulkan instance. The engine then finishes GUI SDL/context shutdown. Owned SDL windows and the SDL video reference are released with WindowSystem.

Why ownership matters

A GPU buffer depends on the allocator and device. An image view depends on its image. A surface depends on the instance and its native window. The code releases dependent resources before the resources they use.

Kind of reference	Lifetime rule
Public World / subsystem wrapper	Use while its Engine is alive.
Internal pointers and index spans	Can become invalid on the documented CPU container mutation or destruction.
Borrowed Vulkan handles	Use while the corresponding owning Vulkan resource remains alive.
Retired GPU resource	Keep until the completion tracking proves earlier submissions are finished.

State survives frames, not destruction
The same objects carry data from one frame to the next. When the engine is destroyed, those objects and their owned resources are released.

Source: [S03], [S04], [S13], [S17], [S31] | Maps: 76-78

Worked example: startup

This application example uses only the public facade and checks the important results.

```
import std;
import VVEngine;

int main(int argc, char** argv) {
    if (argc < 2) return 1; // Supply a model file path.
    auto engine = vve::EngineBuilder<>{}
        .applicationName("Model walkthrough")
        .addWindow(vve::WindowSetup{}
            .title("Model walkthrough")
            .extent(vve::PixelExtent{.width=1280, .height=720}))
        .build();

    if (auto r = engine.init(); !r) return 2;
    auto world = engine.world();
    auto& assets = world.get<vve::AssetSystem>();
    auto& render = world.get<vve::RenderSystem>();

    auto scene = assets.loadScene(std::filesystem::path{argv[1]});
    if (!scene) return 3;
    auto placed = render.instantiateScene(*scene);
    if (!placed) return 4;

    render.setCamera(vve::Camera::lookAt(
        vve::Position{.value=vve::Vec3{0.0f, 3.0f, 8.0f}},
        vve::Position{.value=vve::Vec3{0.0f, 0.0f, 0.0f}}));

    for (;;) {
        const auto status = engine.step();
        if (!status) return 5;
        if (*status == vve::FrameStatus::stopped) break;
    }
    return 0;
}
```

Before the first step, windows and CPU model records exist. The scene instance has draw objects and decoded texture data where needed. The first drawable step performs GPU initialization and uploads before presenting.

The camera is illustrative: choose a position suitable for the model's size and location. Configure lights or opt into imported lights as needed. The example shows the lifecycle and is not a promise that an arbitrary model fits this view.

Source: [S01], [S29], [S33] | Maps: 76-78

Worked example: one object, one frame

Follow the same object through the code without confusing its different identities.

Stage	What the object is now
File	A node references a mesh, a material and an image source.
After loadScene()	Catalog has a Node, AssetMesh and Material, reached through typed source handles.
After instantiateScene()	RenderScene has mesh/material records and a placed RenderInstance. RenderSystem exposes a RenderObjectHandle.
After GPU startup/sync	ForwardRenderer has a VulkanMesh, texture images and packed material data.
During command recording	A DrawItem resolves those resources and supplies a model matrix and material slot.
During GPU execution	Shaders read the buffers, sample textures and write depth/color images.
During presentation	The acquired swapchain image is shown through the window surface.

Useful places to stop in a debugger

Construction: makeEngineImpl(), then the facade constructor. Watch the implementation pointer and the subsystem references.

Import: AssetSystem::loadScene(), materials(), meshes(), node() and import(). Inspect catalog tables and RenderVertex arrays.

Placement: instantiateScene(), acquireRenderMesh() and acquireRenderMaterial(). Inspect shared vertices, instances and texture indices.

GPU setup: ForwardRenderer::init(), createTargetResources() and syncSceneResources(). Watch Vulkan handles becoming valid.

Each frame: facade step(), updateSystems(), drawFrame(), recordCommandBuffer(), vkQueueSubmit() and vkQueuePresentKHR().

> **Stepping on the CPU has a limit**
> A debugger follows the C++ recording and submission code. After vkQueueSubmit, the GPU executes separately. Captures, validation messages, counters and readback help inspect the results.

Source: [S01], [S05], [S09], [S13], [S14], [S26] | Maps: 76-78

Class reference: engine and scene

These classes and records explain the CPU side of the simple implementation.

Class / record family	Plain-language role
Engine	Owns subsystems; handles platform startup, timing and lazy render startup.
Windows	Startup descriptor collection consumed by WindowSystem.
WindowSystem / Impl	Owns SDL video, windows, input and lookup maps. Impl keeps SDL details private.
Window / InputState	Owns one native window / remembers input state.
AssetSystem / Catalog / Table<T>	Imports files / groups tables / stores records by handle.
AssetScene / Node	Imported scene membership and hierarchy / one named transform and mesh list.
AssetMesh / Material	Imported geometry / surface factors and texture sources.
Light / CameraAsset	Imported lighting and camera descriptors.
Graph<T> / Tree<T>	Relationships between node handles; Tree adds a single-root hierarchy.
RenderVertex / EmbeddedImage	Vertex attributes / retained compressed or raw image source.
MaterialTextureSource / MaterialFactors	A texture's role and source / optional imported scalar/color factors.
ImportedAssetReadAccess	Read functions connecting RenderSystem to AssetSystem.
RenderSystem / RenderScene	Coordinates scene requests / owns CPU draw resources and lookups.
RenderMesh / RenderMaterial / RenderInstance	Geometry / appearance / one placed draw object.
RenderTexture / Scene	Decoded texture record and source / renderer lighting collection.
ForwardPointLight / ForwardSpotLight / ForwardDirectionalLight	Renderer-side parameters for the three lighting types.
ForwardRenderer / WindowTarget	Shared GPU owner / one window's rendering and presentation state.
GuiSystem / DrawItem	GUI lifecycle and callback / temporary resolved object data for drawing.

Shared facade types such as Camera, Transform, typed handles and ECS also participate. Empty handle-tag structs name ID categories; they do not own resources. Diagnostic records include FrameDrawStats, ShadowLightMeta and RenderShadowDepthSample.

Source: [S03], [S04], [S05], [S06], [S07], [S08], [S10], [S11], [S12], [S14], [S23] | Maps: 76-78

Class reference: GPU resources

Most Vulkan helpers are small owners around a specific Vulkan resource.

Class / record family	What it owns or describes
VulkanOwnedHandle<...>	A reusable RAII holder for a Vulkan-Hpp resource and its raw handle.
DeviceCapabilities	Queried features used to check whether a GPU meets requirements.
VulkanInstance / VulkanSurface	Loader-level Vulkan instance / native-window presentation surface.
VulkanPhysicalDevice / VulkanDevice	Selected physical GPU and queue families / logical device and borrowed queues.
VulkanAllocator / VulkanBuffer	VMA memory allocator / allocated byte buffer and optional CPU mapping.
VulkanImage / TextureImage / ShadowMap	Allocated image/views / uploaded texture image / shadow depth-array image.
VulkanUploadBatch	Temporary upload commands and staging buffers retained until completion.
VulkanSwapchain / VulkanImageViews	Presentation image owner / image views for drawing and GUI.
VulkanCommandPool / VulkanCommandBuffers	Pool for allocating commands / command buffers allocated from it.
VulkanFrameSync	Slot-indexed acquire semaphores/fences and image-indexed render-finished semaphores.
VulkanDescriptorSetLayout / VulkanDescriptorPool / VulkanDescriptorSets	Binding contract / allocation pool / bound resource-reference sets.
VulkanVertexInputDescription / VulkanPipelineLayout	Vertex byte layout / binding and push-constant contract.
VulkanShaderModule / VulkanGraphicsPipeline	Compiled GPU program / complete graphics drawing configuration.
VulkanMesh / VulkanUniformBuffers	GPU geometry buffers and bounds / mapped per-frame uniform buffers.
FrameUniforms / GpuMaterial / ObjectPushConstants	Packed frame data / packed surface data / small per-object command data.
VulkanReadback	CPU-readable buffer and copy commands for inspecting image results.

BufferMemory chooses allocation intent. AcquireAction chooses what to do with an acquire result. RecordedPass and FrameDrawStats describe observed drawing. Helper functions such as submitOnce(), transitionImage() and allocationInfoFor() carry out actions but are not additional resource owners.

Source: [S12], [S16], [S17], [S18], [S19], [S20], [S21], [S22], [S27], [S31] | Maps: 76-78

Plain-language glossary

Use this page when a Vulkan word interrupts the story.

Word	Meaning in this engine
CPU / GPU	CPU runs C++ game and preparation code; GPU executes uploaded shader programs and drawing/copy commands.
Handle	An identifier used to find or name an object. Public typed IDs and Vulkan handles have different roles.
Owner / reference	An owner keeps a resource alive; a reference or borrowed handle accesses someone else's resource.
Mesh / instance	Mesh is reusable geometry; instance is one placed use of that geometry.
Material / texture / sampler	Appearance parameters / image data / rules for looking up that image.
UV / normal / tangent	Texture position / surface direction / basis direction used for normal mapping.
Transform / matrix	Position, rotation and scale / numbers that convert between local, world, camera and screen spaces.
Buffer / image / view	Byte storage / structured pixel storage / a way to use part or all of an image.
Descriptor / pipeline	A shader's resource references / the programs and settings for a drawing operation.
Command buffer / queue	A recorded instruction list / where lists are submitted for execution.
Surface / swapchain / acquire	Window presentation connection / presentable image set / permission to use one image now.
Fence / semaphore / barrier	CPU completion wait / dependency signal between GPU operations / ordering and visibility inside commands.
Frame slot / image index	Reusable commands/data slot / acquired presentable image. They are not the same index.
Staging / readback	CPU-to-GPU transfer storage / GPU-to-CPU transfer storage.
Dirty / generation / retirement	Needs synchronization / current content identity / delayed release until old GPU use completes.
HDR / depth / shadow map	High-range scene color / nearest surface distance / depths seen from a light.

Source: [S07], [S12], [S17], [S19], [S20], [S21] | Maps: 76-78

Part III: the game programmer guide

Build a first game, then add input, models, ECS, UI, effects and level management.

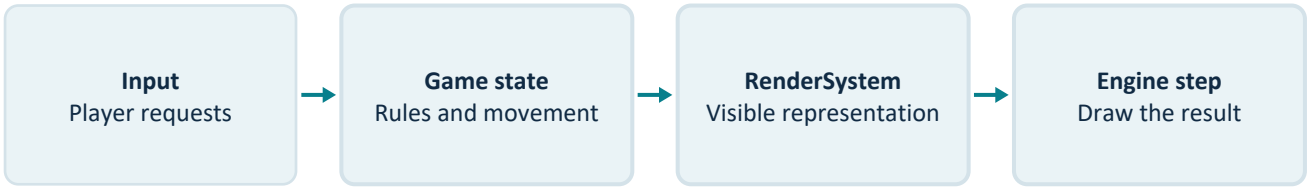
Page	Chapter
52	Your job as a game programmer
53	Build and run your game
54	Starter game: setup and state
55	Starter game: update and main
56	Registering your own game systems
57	Time, update order and your game loop
58	Keyboard, mouse and window input
59	Cameras and player movement
60	Procedural objects and tiled ground
61	Moving objects and editing geometry
62	Loading and placing model files
63	Finding parts of an imported model
64	Entities and components
65	Connecting ECS data to rendering
66	Gameplay, collision and fixed steps
67	Lighting and shadows for a game
68	HUDs, menus and GUI state
69	Post-processing and effect controls
70	Multiple windows and their cameras
71	Levels, removal and resource lifetimes
72	Keeping a game responsive
73	Errors, captures and debugging
74	The crate game, step by step
75	Public API task reference

Pages 54-55 form one complete starter source file. Other listings are focused excerpts. Newly written snippets were source-checked but not compiled or run separately.

Your job as a game programmer

The engine provides the window and picture; your game provides the rules and state.

Use **import VVEngine;** and the **vve** namespace. Obtain subsystems through engine.world().get<T>(). This is the supported application interface. Internal vve::simple classes explain implementation details in Part II, but are not the interface for a game.



The engine supplies	Your game supplies
Window creation, events and input queries	Key bindings, player intent and pause/menu behavior
Elapsed time and optional user-system hooks	Simulation, scoring, spawning and win/lose rules
Imported asset data and render-object handles	Level organization and the mapping from gameplay to objects
Cameras, lights, shadows and GPU drawing	Camera behavior, lighting design and scene placement
An ECS component store and GUI callback	Component types, systems, HUD contents and saving game state

Start with a small vertical slice

First show one cube on a ground plane. Add a camera and a light. Move the cube using fresh input in a registered system. Then add one gameplay rule, such as collecting a crate. A small working loop gives you a place to verify assets, timing and object ownership before adding a full level.

Keep game state separate from drawing state

A crate needs position, falling speed and a collected flag. Its RenderObjectHandle only identifies the visible object. Rendering does not calculate gravity or decide when the crate has been collected. Keep those values in a game struct or ECS components, then send the resulting transform to RenderSystem.

> **Scope of the current engine**
> This public facade has no general physics solver, audio API, animation player, networking API or save-game service. Build game rules around the exposed facilities or integrate separate game libraries. Loading a file is not a promise that every feature in that file is supported.

Source: [S01], [S32], [S37], [S39], [S34] | Maps: 76-78

Build and run your game

Use the repository build setup so C++ modules, dependencies and generated shaders agree.

Run the existing game first

On Windows, open a Developer PowerShell for Visual Studio in the repository root. The supplied scripts and presets use Ninja because this project depends on C++ modules and import std. Build the debug variant to step through ordinary C++ code with symbols.

```
.\build_windows.cmd debug
.\bin\debug\exe\game.exe

# Alternatively, after prerequisites are available:
cmake --preset debug-windows
cmake --build build/debug-windows --target game
ctest --test-dir build/debug-windows --output-on-failure
```

The first command also runs the repository tests. The alternative commands separate configuration, building and testing. The expected example executable is bin/debug/exe/game.exe; build files are in build/debug-windows. Release changes the variant directories and removes much of the convenient stepping behavior.

Add an application target inside this configured project

```
add_executable(my_game my_game.cpp)
target_link_libraries(my_game PRIVATE
    ViennaVulkanEngine::ViennaVulkanEngine)
add_dependencies(my_game vve_simple_shaders)
vve_set_output_dirs(my_game)
```

This CMake excerpt assumes it is processed after the engine and helper functions are defined. Add its directory with add_subdirectory() or follow examples/game/CMakeLists.txt. A separate parent project must include the engine with its required toolchain and dependencies; this small excerpt is not a complete standalone configuration.

Use the existing VS Code launch configuration

Open the repository folder, select **Windows Debug (choose executable)** in Run and Debug, and start it. The input prompt offers game and the other configured executables. Its debugger type is cppvsdbg, and its working directory is the repository root. For a new target, add the name to the executable input list or set program directly.

> **Paths belong to your application**
> Resolve content from a chosen asset root, not from whichever directory happened to launch the program. VVE.ExampleSupport and assetRoot() are example helpers; link vve_example_support to use them. A standalone game can take an asset directory as an argument instead.

Source: [S44], [S45], [S46], [S48] | Maps: 76-78

Starter game: setup and state

Start one source file here, then continue it on the next page.

This starter needs no image files. It creates a green plane and a cube, moves the cube with A/D, and lets Escape end the application. All scene creation happens in the game system init hook, after windows exist and before the first GPU frame.

```
import std;
import VVEngine;

struct MoveCubeGame {
    vve::RenderObjectHandle cube{};
    vve::Position position{.value = vve::Vec3{0, 0.5F, 0}};
    bool quit{};

    std::expected<void, vve::Error> init(auto& world) {
        auto& render = world.template get<vve::RenderSystem>();
        auto floor = render.addPlane(vve::Vec2{5, 5},
            vve::LinearColor{.value = vve::Vec3{0.1F, 0.5F, 0.2F}});
        if (!floor) return std::unexpected(floor.error());
        auto created = render.addCuboid(
            vve::Vec3{-0.5F, -0.5F, -0.5F},
            vve::Vec3{0.5F, 0.5F, 0.5F},
            vve::LinearColor{.value = vve::Vec3{0.8F, 0.4F, 0.1F}},
            vve::Transform{.translation = position});
        if (!created) return std::unexpected(created.error());
        cube = *created;
        render.setCamera(vve::Camera::lookAt(
            vve::Position{.value = vve::Vec3{0, 5, 8}},
            vve::Position{.value = vve::Vec3{0, 0.5F, 0}}));
        render.setDirectionalLight(
            vve::Direction{.value = vve::Vec3{0.3F, -1, 0.2F}},
            vve::LinearColor{.value = vve::Vec3{1, 1, 1}},
            vve::LightIntensity{.value = 1},
            vve::LinearColor{.value = vve::Vec3{0.15F, 0.15F, 0.15F}});
        return {};
    }
    // Continue with update() on the next page.
```

cube is a drawing handle; position and quit are gameplay state. The floor can remain in the render scene without keeping a handle if you will not manipulate it separately. In a larger game, keep all level-owned handles so you can unload a level deliberately.

The local cube spans -0.5 to +0.5 in each axis. Translating its center to y=0.5 makes its bottom touch the XZ floor at y=0. The camera looks at the cube and the light points toward the scene.

Source: [S01], [S29], [S37] | Maps: 76-78

Starter game: update and main

Append this to the previous page to complete the single source file.

```
    std::expected<void, vve::Error> update(
        auto& world, const vve::FrameContext& frame) {
        auto input = world.template get<vve::WindowSystem>().input();
        quit = quit || input.wasKeyPressed(vve::Key::escape);
        if (quit) return {};
        const auto dt = static_cast<vve::Scalar>(
            std::clamp(frame.delta_time.seconds, 0.0, 0.1));
        vve::Scalar direction{};
        if (input.isKeyDown(vve::Key::a)) direction -= 1;
        if (input.isKeyDown(vve::Key::d)) direction += 1;
        position.value.x = std::clamp(
            position.value.x + direction * 3 * dt,
            vve::Scalar{-4}, vve::Scalar{4});
        return world.template get<vve::RenderSystem>()
            .setObjectTransform(cube,
                vve::Transform{.translation = position});
    }
};

int main() {
    auto engine = vve::EngineBuilder<MoveCubeGame>{}
        .applicationName("My first game")
        .addWindow(vve::WindowSetup{}.id("main")
            .title("A/D move - Escape quits")
            .extent(vve::PixelExtent{.width = 960, .height = 540}))
        .build();
    if (auto result = engine.init(); !result) {
        std::cerr << vve::errorName(result.error()) << '\n';
        return 1;
    }
    while (!engine.world().get<MoveCubeGame>().quit) {
        const auto status = engine.step();
        if (!status) {
            std::cerr << vve::errorName(status.error()) << '\n';
            return 2;
        }
        if (*status == vve::FrameStatus::stopped) break;
    }
    return 0;
}
```

What happens when you press D

step() polls input, then calls MoveCubeGame::update(). A held key changes position by speed times seconds. Rendering uses the new transform while reusing the same GPU mesh.

> **Why main() checks quit**
> The facade has no requestStop(). FrameStatus::stopped reports engine close/cap conditions. Your quit flag is separate: set it in update, then let main stop stepping.

Source: [S01], [S32], [S36], [S37] | Maps: 76-78

Registering your own game systems

A system is an ordinary game object whose optional hooks the engine recognizes.

Choose your system types in Engine<TSystems...> or EngineBuilder<TSystems...>. The engine stores actual system objects by value. Calls occur in the order of the template types and supplied tuple. These are direct C++ calls, not a global system registry.

Hook	When and how it is used
init(world)	After platform/window initialization. Optional. Missing init means no work is needed.
update(world, frame, window_frame)	Highest priority update form. Receives timing plus current window-state copies.
update(world, frame)	Selected if the three-argument form is unavailable. Good default for game simulation.
update(world)	Selected if the timed forms are unavailable. Suitable for work that does not use time.
No update hook	The system remains accessible state; the engine performs no update call on it.

Only the highest matching update form runs for a system. Prefer void for hooks that cannot fail, and std::expected<void, vve::Error> for hooks that can. Only that exact expected return type is propagated as an engine error; returning false does not signal failure to the scheduler.

```
struct ScoreState { int points{}; };
struct MovementSystem {
    void update(auto& world, const vve::FrameContext& frame) {
        auto& score = world.template get<ScoreState>();
        // Use frame.delta_time.seconds and facade subsystems here.
    }
};
auto engine = vve::EngineBuilder<ScoreState, MovementSystem>{}.build();
```

Supplying state and sharing it

makeUserSystems(...) packages configured objects for a matching typed builder or Engine. Use world.get<YourType>() to change the stored object; editing an original copied into it does not change this instance. Avoid repeated types: get<T>() selects the first match.

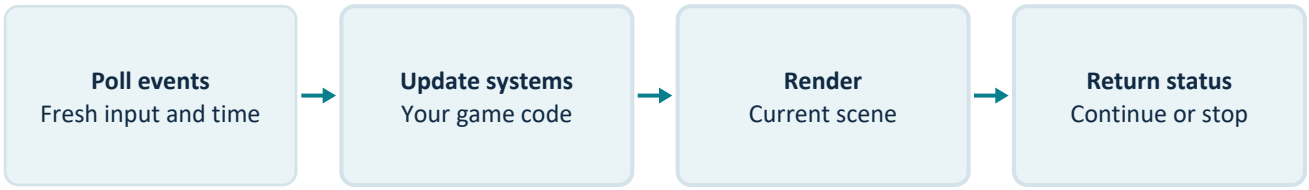
build() copies its systems. For move-only state, construct Engine directly with a temporary makeUserSystems(...) bundle. Inside an auto& world hook, use world.template get<T>(); outside templates, use world.get<T>().

> **Initialization retries**
> If one init hook fails, earlier successful hooks are not repeated on the next init() attempt. The failed hook may run again, so it must clean up partial work or make repeated work safe. A failed update stops later updates and rendering for that step.

Source: [S01], [S32] | Maps: 76-78

Time, update order and your game loop

Pick one owner for update timing, and know exactly where input and drawing happen.



Use the supplied frame context

frame.delta_time.seconds is elapsed time between engine polling steps. The first delta is 1/60 second, and frame.frame_index.value begins at zero. Multiply speeds by elapsed seconds. A speed of 3 world units per second should move 0.05 units in a 1/60-second update, not 3 units per displayed frame.

For a small interactive game, clamp unusually long deltas after pauses or debugging, as the starter does. That deliberately discards some elapsed time; it prevents a huge movement jump, but is not a deterministic simulation method. The fixed-step recipe on page 66 describes a more controlled approach.

Loop choice	Useful behavior and limitation
engine.run()	Calls init(), then step() until the engine stops or returns an error. Convenient for games using only engine close/cap conditions.
Explicit engine.step() loop	Lets main inspect errors, game quit state, external tasks or a custom frame limit. Call init() before stepping.
Game logic before step()	Uses previously polled input, then step polls again and draws. Existing examples use this style in places.
Game logic after step()	Can see freshly polled input, but its changes appear on the next step because drawing already happened.

Prefer hooks for input-to-picture work

Registered updates sit between polling and rendering, so one movement response can affect the same step. Do not call engine.step() recursively from a system. Keep main responsible for advancing the engine, and keep systems responsible for modifying gameplay and scene state.

MaxFrames limits step calls. It is not an FPS setting or a sleep mechanism. A step may skip drawing for minimized windows or an acquisition timeout. With multiple windows, one step can draw several pictures. Engine frame index and renderedFrameCount() therefore measure different events.

> **The final update still runs**
> A successful backend step can report stopped and still be followed by user updates and a final rendering attempt before the facade returns stopped. Do not assume stopped means no update occurred. Break your main loop when you receive it.

Source: [S01], [S03], [S29], [S34] | Maps: 76-78

Keyboard, mouse and window input

Read held keys for continuous actions and edge events for actions that should happen once.

```
// Inside update(auto& world, ...), after engine polling:
auto input = world.template get<vve::WindowSystem>().input();
if (input.isKeyDown(vve::Key::w)) { /* keep moving */ }
if (input.wasKeyPressed(vve::Key::p)) { /* toggle pause */ }
if (input.wasKeyReleased(vve::Key::w)) { /* stop a held action */ }
```

Query	Meaning
isKeyDown(key)	The key remains held across steps until released.
wasKeyPressed(key)	A press edge in the most recently polled step; repeats do not require repeated toggling.
wasKeyReleased(key)	A release edge in that step.
mousePosition(window)	Optional normalized position: top left (0,0), bottom right (1,1), y increasing downward.
mouseDelta(window)	Normalized movement accumulated for that window in the current poll.
mouseWheelDelta(window)	Accumulated wheel movement in scroll ticks.

Identify the correct window

```
auto& windows = world.template get<vve::WindowSystem>();
if (auto main = windows.findWindow("main")) {
    const auto window = main->handle();
    const auto cursor = input.mousePosition(window);
    const auto motion = input.mouseDelta(window);
    // cursor may be absent; motion uses normalized window units.
}
```

Drags can leave 0..1. Deltas are normalized, not pixels: scale horizontal movement by aspect ratio for matching mouse-look angles. Keyboard state is shared, not per-window; use focus and game mode to decide which view accepts control.

Treat InputState as a live view

Copying InputState copies a reference, not historical values. After another step, queries see the new poll. Copy specific key/mouse values into your own action record for simulation or replay.

> **GUI interaction**
> Claimed key-down and mouse events can be withheld from game input; releases still pass through. Use a gameplay/menu mode too. Engine polling owns beginFrame(), pressKey() and releaseKey(); game code normally only queries input.

Source: [S36], [S04], [S23] | Maps: 76-78

Cameras and player movement

A camera describes a view. A controller is game code that changes that view.

```
auto camera = vve::Camera::lookAt(
    vve::Position{.value = vve::Vec3{0, 3, 8}},
    vve::Position{.value = vve::Vec3{0, 1, 0}});
camera.fov_y = vve::FovY{.radians = 1.0472F}; // About 60 degrees.
camera.clip = vve::ClipPlanes{.near_plane = 0.1F, .far_plane = 100};
render.setCamera(camera); // render is your public RenderSystem view.
```

The practical convention used by these examples is Y up, X left/right and Z depth; the default camera faces -Z. Camera::lookAt stores position and the direction to the target. Its up argument is currently retained only for call compatibility; it does not provide arbitrary camera roll. Avoid a zero look direction or a view exactly parallel to world up.

Use the built-in keyboard controller

```
// Store controller as a member of your game system:
vve::DefaultCameraController controller{};

// In update(auto& world, const vve::FrameContext& frame):
auto input = world.template get<vve::WindowSystem>().input();
world.template get<vve::RenderSystem>().setCamera(
    controller.update(input, frame.delta_time));
```

Control	Current controller behavior
W / S	Move forward/backward along its look direction, including vertical pitch.
A / D	Move left/right; Q/E move vertically.
Arrow keys	Change yaw and pitch; pitch is clamped.
Shift	Double movement speed.
eye, yaw, pitch, move_speed, turn_speed	Public controller state you can set and keep between steps.

For a walking player

The existing crate game resets the camera eye to a fixed height before and after controller.update(). This prevents vertical displacement, but forward walking speed still depends on pitch. For constant ground speed, build and normalize a horizontal XZ direction yourself, then integrate movement using that direction. Keep player collision separate from camera drawing.

Choose clip distances to fit your level. The default far plane is 100 world units, not infinity. The renderer derives projection aspect from each window and also uses clip distances for directional shadow coverage. Imported camera ownership alone does not select a view; call setCamera() explicitly.

Source: [S29], [S36], [S40], [S09] | Maps: 76-78

Procedural objects and tiled ground

Create shapes once, keep their handles, and let the renderer reuse their resources.

Creation call	What the arguments describe
addPlane(half_extent, color, transform)	A local XZ ground quad. Half extents {5,3} give a full 10 by 6 plane.
addCuboid(minimum, maximum, color, transform)	Local corners, surface color and placement. {-0.5...+0.5} gives a unit cube.
addTexturedPlane(half_extent, path, uv_scale, transform)	One XZ quad with a base-color image and UV repetition.
addTexturedCuboid(minimum, maximum, path, transform)	A cuboid with the given image as its base-color texture.
addTriangleMesh(positions, indices, color, transform)	A custom indexed triangle shape. Indices must be valid and come in triples.

A repeated texture on one plane

```
const auto grass = assetRoot / "assets/game/plane/grass.jpg";
auto ground = render.addTexturedPlane(
    vve::Vec2{20, 20}, grass, vve::Vec2{10, 10});
if (!ground) return std::unexpected(ground.error());
```

This creates a 40 by 40 ground plane. UVs span 0..10 in each axis. REPEAT addressing wraps the image outside 0..1, so the grass repeats ten times in each direction. A tile occupies 4 by 4 world units. There are no tiled ground boxes: the plane is one quad made of two triangles.

For a desired tile width t and full plane width w, use w/t repeats. If you double plane size and want the same apparent grass size, double uv_scale too. Changing a transform scale stretches the texture with the geometry unless you author matching UV repeats.

Creating custom geometry

Pass vve::Vector<vve::Vec3> positions and vve::Vector<std::uint32_t> indices by value, normally with std::move when you are done building them. addTriangleMesh() is a colored position/index convenience API; it is not a complete custom vertex/material/shader authoring interface.

> **Texture and material control**
> The current facade exposes texture paths at object creation and imports material properties from model files. It does not expose an arbitrary setObjectMaterial() call, a per-object sampler setting, or a general textured custom-vertex call. Avoid designing game features around missing calls.

Source: [S37], [S28], [S34] | Maps: 76-78

Moving objects and editing geometry

For ordinary movement, change transforms. Edit vertex positions only for deformation.

```
// In a function returning std::expected<void, vve::Error>:
auto transform = render.objectTransform(object);
if (!transform) return std::unexpected(transform.error());
transform->translation.value.x += 1;
auto moved = render.setObjectTransform(object, *transform);
if (!moved) return std::unexpected(moved.error());
```

Transform stores translation, quaternion rotation and scale, defaulting to zero, identity and one. Keep the full transform or edit the returned value. Transform{.translation=...} resets rotation and scale to their defaults.

Rotate using the public math types

```
const vve::Scalar angle = 0.5F; // Radians around Y.
transform->rotation.value = vve::Quat{
    std::cos(angle / 2), 0, std::sin(angle / 2), 0};
transform->scale.value = vve::Vec3{2, 1, 2};
auto changed = render.setObjectTransform(object, *transform);
if (!changed) return std::unexpected(changed.error());
```

This quaternion constructor is w, x, y, z. Use normalized rotations. Transform animation reuses meshes/textures and records a new model matrix: use it for rigid crates, doors, platforms and projectiles.

Object operation	Effect on the game
setObjectVisible(handle, bool)	Hide/show an existing object without losing its handle.
setObjectCastsShadow(handle, bool)	Include/exclude it from shadow passes.
setObjectUnlit(handle, bool)	Use flat base color/texture instead of scene lighting.
removeObject(handle)	Delete the render object. Remove its handle from your own live records too.
setObjectMeshPositions(handle, positions)	Replace all positions with the same count and finite coordinates; indices remain unchanged.

Geometry editing has narrower rules

Position editing detaches shared geometry and updates safe GPU vertex copies. It does not rebuild normals/tangents, so large deformations can shade incorrectly. New triangles require a new mesh.

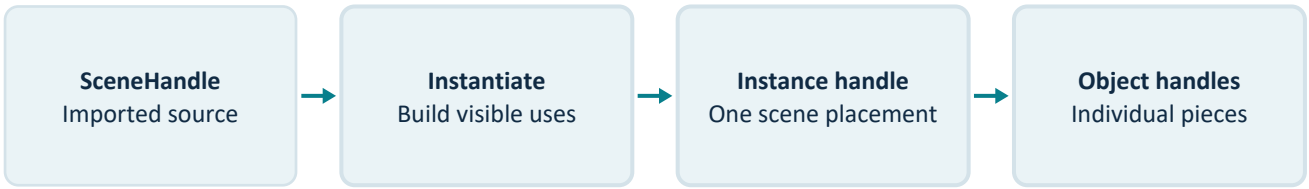
> **Imported objects need special care**
> Instantiation composes the asset hierarchy into world matrices. setObjectTransform() replaces one object's world placement and moves nothing else. To move a whole model, or a part with everything below it, instantiate with keep_hierarchy and set node transforms (page 63). A direct placement then holds until its node or an ancestor is set again.

Source: [S29], [S35], [S28], [S09], [S49], [S55] | Maps: 76-78

Loading and placing model files

Import the source scene, instantiate it into the drawing scene, then keep both handles.

```
// render and assets are public subsystem views.
const auto path = assetRoot / "assets/my_level/scene.gltf";
auto source = assets.loadScene(path);
if (!source) return std::unexpected(source.error());
auto placed = render.instantiateScene(*source,
    vve::SceneInstantiationOptions{
        .instantiate_geometry = true,
        .apply_cameras = false,
        .apply_lights = true,
        .keep_hierarchy = false});
if (!placed) return std::unexpected(placed.error());
// Store *source and *placed in your level's state.
```



Handle kind	What it identifies
SceneHandle	Imported CPU asset content in AssetSystem. loadScene() alone does not show it.
RenderSceneInstanceHandle	One placed group of objects, optional imported light/camera ownership and its node tree with keep_hierarchy.
RenderObjectHandle	A single live drawable use of a mesh and material.
MeshHandle / NodeHandle / MaterialHandle	Parts of the imported source, used for inspection and source lookup.

Default options instantiate geometry but do not apply imported lights or cameras and do not keep the node tree. Applying cameras currently records ownership/counts rather than selecting the renderer view. Query camera data and call setCamera() if you want a particular imported viewpoint. Imported texture decode failures may leave a fallback material instead of failing the whole placement.

Plan an asset pipeline

Export a simple static test scene first. Verify dimensions, orientation, triangle counts, texture UVs and relative image paths. Assimp reads the model data; it does not make every imported extension available to your game. This facade has no skeletal animation playback or automatic script/component creation from imported nodes.

Loading can block and the first visible use may upload to the GPU. Load content during initialization or a deliberate loading transition. Reusing the same source scene for multiple instances allows render meshes/materials to be shared. To move a complete imported group, set keep_hierarchy and move its root node (page 63).

Source: [S38], [S37], [S09], [S33], [S55] | Maps: 76-78

Finding parts of an imported model

Map named source nodes to objects, then move parts through the node tree.

```
// In a function returning std::expected<void, vve::Error>:
auto objects = render.sceneInstanceObjects(instance);
if (!objects) return std::unexpected(objects.error());
for (auto object : *objects) {
    auto node = render.objectSourceNode(object);
    if (!node) return std::unexpected(node.error());
    auto name = assets.nodeName(*node);
    if (!name) return std::unexpected(name.error());
    if (name->value == "Door") {
        // Save object in your Door game record.
    }
}
```

A node is not always one drawable object

A source node can have zero, one or several meshes, so an instance can hold several render objects for one named node. Store all of them, give exported parts stable names and handle duplicate names explicitly.

Inspection goal	Public queries
Find scene content	sceneRootNode, sceneNodes, sceneMeshes, sceneMaterials, sceneLights, sceneCameras
Follow relationships	sceneNodeChildren(scene,node), sceneNodeParent(scene,node)
Inspect a node	nodeName, nodeTransform, nodeMeshes, nodeMaterials
Inspect geometry	meshBounds, meshPositions, meshNormals, meshTexcoords, meshIndices and meshMaterial
Inspect imported view/light data	cameraData(CameraHandle), lightData(LightHandle)
Trace a drawable back	objectSourceScene and objectSourceNode for imported render objects

Asset queries return owning values and copied arrays; use them during setup. The engine creates no gameplay from a node name: your game supplies interaction, collision and state.

Move a part and everything below it

```
auto car = render.instantiateScene(*source,
    vve::SceneInstantiationOptions{.keep_hierarchy = true});
if (!car) return std::unexpected(car.error());
auto moved = render.setSceneInstanceNodeTransform(*car, node, local);
if (!moved) return std::unexpected(moved.error());
```

Here node is the named part's NodeHandle (*node above) and local its new parent-relative Transform; nodeTransform() gives the imported one. The node's objects and all objects below it follow, even through mesh-free nodes; imported lights and cameras do not. examples/hierarchy_car drives a car this way.

Source: [S38], [S37], [S09], [S35], [S55], [S57] | Maps: 76-78

Entities and components

Use entities for gameplay identity and components for the data each behavior needs.

The public ECS is a small typed component store. An Entity identifies a game record. A component can be your own ordinary C++ struct. A render object is separate: ECS does not automatically create or draw it. Store the RenderObjectHandle as a component when you want to connect the two.

```
struct Velocity { vve::Vec3 value{}; };
struct Drawable { vve::RenderObjectHandle object{}; };

// ecs and render are public subsystem references; this function
// returns std::expected<void, vve::Error>.
const auto entity = ecs.create();
auto cube = render.addCuboid(vve::Vec3{-0.5F, -0.5F, -0.5F},
    vve::Vec3{0.5F, 0.5F, 0.5F},
    vve::LinearColor{.value = vve::Vec3{1, 0.5F, 0.2F}});
if (!cube) { (void)ecs.erase(entity); return std::unexpected(cube.error()); }
auto result = ecs.add(entity, vve::Transform{});
if (!result) return std::unexpected(result.error());
result = ecs.add(entity, Velocity{.value = vve::Vec3{1, 0, 0}});
if (!result) return std::unexpected(result.error());
result = ecs.add(entity, Drawable{.object = *cube});
if (!result) return std::unexpected(result.error());
```

ECS operation	Behavior
create / exists / erase	Create identity, test it, or delete the entity and all its components.
add<T>	Insert once; duplicate type returns duplicate_component.
get<T> / tryGet<T>	Copy a required component / copy an optional component. Invalid entities still fail.
put<T>	Insert or replace a component on an existing entity.
has<T> / remove<T>	Check a component or remove that component type.
view<T...>	Return entity handles that contain every requested component type.

Copies must be written back
get() returns an expected containing a component copy, not a reference into the ECS. Editing it does not modify stored state. Call put(entity, changed_component) to commit the change.

The creation excerpt shows the API, but a reusable spawn function also needs rollback for every later failure. Keep the newly created render handle and erase it if component setup fails. Avoid leaving an entity half configured or a drawable without gameplay ownership.

Source: [S39], [S37], [S30] | Maps: 76-78

Connecting ECS data to rendering

Use one system to change gameplay state and another to publish that state to RenderSystem.

```
struct IntegrateMovement {
    std::expected<void, vve::Error> update(
        auto& world, const vve::FrameContext& frame) {
        auto& ecs = world.template get<vve::ECS>();
        const auto dt = static_cast<vve::Scalar>(
            std::clamp(frame.delta_time.seconds, 0.0, 0.1));
        for (auto entity : ecs.template view<vve::Transform, Velocity>()) {
            auto t = ecs.template get<vve::Transform>(entity);
            auto v = ecs.template get<Velocity>(entity);
            if (!t) return std::unexpected(t.error());
            if (!v) return std::unexpected(v.error());
            t->translation.value = vve::math::add(t->translation.value,
                vve::math::scale(v->value, dt));
            if (auto r = ecs.put(entity, *t); !r) return r;
        }
        return {};
    }
};
struct PublishDrawables {
    std::expected<void, vve::Error> update(auto& world) {
        auto& ecs = world.template get<vve::ECS>();
        auto& render = world.template get<vve::RenderSystem>();
        for (auto entity : ecs.template view<vve::Transform, Drawable>()) {
            auto t = ecs.template get<vve::Transform>(entity);
            auto d = ecs.template get<Drawable>(entity);
            if (!t) return std::unexpected(t.error());
            if (!d) return std::unexpected(d.error());
            if (auto r = render.setObjectTransform(d->object, *t); !r)
                return r;
        }
        return {};
    }
};
```

These excerpts use the Velocity and Drawable structs from the previous page. Register IntegrateMovement before PublishDrawables. If you reverse them, drawing receives the previous position for a step. Add input and gameplay systems before publishing so all changes become visible together.

view() returns a copied list of entity handles. Keep component reads/writes deliberate and gather pending removals if a system will delete entities used by another phase. Vector growth may invalidate your own references; prefer handles and lookups across mutations.

> **Deletion crosses two stores**
> ecs.erase(entity) removes gameplay components, but does not call removeObject() on the renderer. Remove the Drawable object deliberately and then erase the entity, or use a destruction queue with a clear ownership policy.

Source: [S39], [S01], [S37] | Maps: 76-78

Gameplay, collision and fixed steps

The renderer shows motion; your simulation calculates it and decides interactions.

Simple falling and collection

Keep each crate's position, vertical velocity and landed flag. For each simulation step: subtract gravity times seconds from velocity, add velocity times seconds to height, then clamp the center to the crate's resting height when it reaches the plane. Publish the resulting transform. This is a simple game-specific integration rule, not a rigid-body solver.

```
velocityY -= gravity * dt;
position.y += velocityY * dt;
if (position.y <= halfHeight) {
    position.y = halfHeight;
    velocityY = 0;
    landed = true;
}
// Horizontal proximity collection test:
const auto dx = player.x - position.x;
const auto dz = player.z - position.z;
const bool collected = landed && dx*dx + dz*dz <= radius*radius;
```

That proximity test ignores height and obstacles. Use it when it matches your game rule. For a platformer or a vehicle, maintain collision volumes, broad-phase candidates and appropriate resolution in game code or a separate physics library. GPU depth tests and shadows do not provide player collision.

Fixed simulation steps, variable drawing

```
// Game-system members:
double accumulator{};
static constexpr double fixedSeconds = 1.0 / 60.0;
// In the timed update hook (algorithm excerpt):
accumulator += std::clamp(frame.delta_time.seconds, 0.0, 0.25);
int steps{};
while (accumulator >= fixedSeconds && steps < 8) {
    simulate(fixedSeconds); // Your gameplay function.
    accumulator -= fixedSeconds;
    ++steps;
}
publishTransforms(); // Your RenderSystem calls, once per engine step.
```

At the catch-up limit, explicitly retain backlog, discard excess time or pause with a diagnostic. Optional interpolation uses previous/current states and the remaining time fraction. Keep one input action snapshot across catch-up; consume a press edge once, not once per substep.

> **Pause and reproducibility**
> Pause gameplay updates while allowing input and GUI updates to continue. For repeatable tests/replays, use fixed time, a chosen random seed and recorded actions. Clamping a variable delta alone does not make a deterministic simulation.

Source: [S34], [S01], [S36] | Maps: 76-78

Lighting and shadows for a game

Choose a few useful lights and verify their direction, range and shadow cost.

```
render.clearLights();
render.setDirectionalLight(vve::DirectionalLight{
    .direction = vve::Direction{.value = vve::Vec3{0.3F, -1, 0.2F}},
    .color = vve::LinearColor{.value = vve::Vec3{1, 0.96F, 0.85F}},
    .intensity = vve::LightIntensity{.value = 1.3F},
    .ambient = vve::LinearColor{.value = vve::Vec3{0.15F, 0.15F, 0.15F}}});
render.addPointLight(vve::PointLight{
    .position = vve::Position{.value = vve::Vec3{2, 3, 0}},
    .color = vve::LinearColor{.value = vve::Vec3{1, 0.4F, 0.1F}},
    .intensity = vve::LightIntensity{.value = 3},
    .range = vve::LightRange{.value = 8}});
```

Light choice	Use and current behavior
Directional light	Sunlight-like direction from the light toward the scene; four shadow cascades per light.
Point light	Local source shining in all directions; six shadow faces per light.
Spotlight	Position, direction, range and cone angle in radians; one shadow view per light.
set...Light	Replace the list of that light type with one light. Other types remain.
add...Light / clearLights	Append a light within capacity / remove all lights without removing geometry.

The current caps are 10 directional, 10 point and 10 spot lights. Addition methods return void and ignore excess additions rather than returning a capacity error. Counts are available for checking what was retained. Shadows can multiply drawing work substantially; start with one directional light and add local sources only where they help.

Objects that represent a light source

A glowing sun sphere is a mesh, not a light by itself. Use setObjectUnlit() so its color is independent of scene shading, and disable its shadow casting. Create the actual light separately. This is how the crate game renders its visible sun and its directional illumination.

LinearColor describes linear values. Base-color images are sampled as sRGB, while data maps use linear sampling. The current ambient implementation stores the red component as a scalar; using different ambient RGB components does not give an independent colored ambient light. Use equal components when choosing a predictable ambient level.

> **Imported lighting and later changes**
> apply_lights can add imported lights tagged to a scene instance. A subsequent setDirectionalLight() replaces that directional list. Removing the instance removes its surviving owned lights. There are no public per-light handles for arbitrary edits/removals; manage lists and descriptors in game state.

Source: [S37], [S35], [S47], [S34] | Maps: 76-78

HUDs, menus and GUI state

Register a callback that builds the HUD each frame; keep its captured state alive.

Dear ImGui declarations are ordinary headers. Put #include <imgui.h> before module imports in a normal game translation unit. The public GuiSystem::draw() stores a callback; it does not immediately run it or begin a separate game loop.

```
// At the top of the file, before import declarations:
#include <imgui.h>

// After engine.init(), using a game system stored by the engine:
auto& game = engine.world().get<MoveCubeGame>();
engine.world().get<vve::GuiSystem>().draw([&game] {
    ImGui::Begin("Game");
    ImGui::Text("Cube x: %.2f", static_cast<double>(game.position.value.x));
    if (ImGui::Button("Quit")) game.quit = true;
    ImGui::End();
});
```

The example uses the engine-owned MoveCubeGame from pages 54-55. Its lifetime covers the callback; a helper's local temporary would not. draw() replaces the single callback, so compose all panels inside it.

Keep simulation out of UI drawing

Use the callback to show score, expose settings and record requests such as pause, restart or quit. Apply larger state transitions in a game update. This keeps gameplay rules independent of whether the GUI produces vertices, whether rendering is skipped, and how many windows exist. If UI changes occur after updates, they may be consumed on the next step.

Always balance ImGui::Begin() with End(), including when Begin returns false. For menu input, use your game mode together with the GUI's event capture. A control panel taking keyboard input should not simultaneously move the player.

Lifecycle fact	Practical consequence
GUI starts with the renderer	Register the callback early; do not call ImGui drawing functions outside a prepared frame.
First rendered target owns the GUI pass	Additional windows do not automatically receive their own HUD callbacks.
GUI setup can fail and disable UI	Keep console diagnostics and a way to close the application.
Caught std::exception in callback	Logged/recovered during preparation; next step reports platform_error.

A restart request needs a level owner
A Restart button should set a flag. The next update removes level-owned objects, resets timers/score and creates the level again. Avoid doing partial resource teardown inside an ImGui draw callback.

Source: [S41], [S23], [S34] | Maps: 76-78

Post-processing and effect controls

Configure an effect chain before the first drawable step, then change its settings from game code.

RenderSystem::setPostProcessSetup() takes a callback receiving vvppl::PostProcessing. VVPPL is the linked post-processing library. Include <VVPPL.h> before imports when using its concrete declarations, as the repository postprocessing example does.

```
// Keep settings storage alive throughout engine rendering:
struct EffectState { vvppl::TonemapSettings* tonemap{}; } effects{};
// effects lives until after the loop; render is a facade view.
render.setPostProcessSetup([&effects](vvppl::PostProcessing& pp) {
    auto& settings = pp.addTonemap();
    settings.exposure = 0.6F;
    effects.tonemap = &settings;
});
// Later, after at least one successful rendering initialization:
if (effects.tonemap) effects.tonemap->exposure = 0.9F;
```

Understand the time of each operation

Setup is deferred until lazy GPU startup creates each window's chain. The pointer stays null until then. Retained settings change that chain; registering setup again is not a documented rebuild operation.

Effects process HDR before GUI. Without a chain, the renderer copies/blits HDR to the swapchain; tone mapping is not automatic. An empty configured chain still uses library apply().

Use the repository example as a settings reference

examples/postprocessing/postprocessing.cpp shows addTonemap, addChromatic, addGreyscale, addVignette and addFilmGrain with retained settings, plus an --all-effects path covering the current library effect list. Follow those actual APIs rather than inventing an effect method from its visual name. Effect order is the order you append to the chain.

Design issue	How to handle it
Pointer lifetime	Treat settings references as owned by the chain and valid through normal resizes until engine shutdown.
Several windows	Setup can run once for each target. A single stored pointer is overwritten by the last callback; store one entry per callback if you need all settings.
Deferred callback captures	Capture game/effect state that outlives rendering, not a helper's local variables.
GUI controls for effects	Only read/edit a settings pointer after checking it is initialized.

Keep game rules independent of effects
An effect should change how the picture looks. A gameplay event can adjust exposure or another retained setting, but scoring and simulation should still function when no post-processing chain is configured.

Source: [S37], [S43], [S13], [S14] | Maps: 76-78

Multiple windows and their cameras

Configure windows at startup, then choose a camera for each rendered window.

```
auto engine = vve::EngineBuilder<>{}
    .addWindow(vve::WindowSetup{}.id("main").title("Player")
        .extent(vve::PixelExtent{.width = 960, .height = 540}))
    .addWindow(vve::WindowSetup{}.id("overview").title("Overview")
        .extent(vve::PixelExtent{.width = 640, .height = 480}))
    .build();
if (auto r = engine.init(); !r) return 1;
auto render = engine.world().get<vve::RenderSystem>();
auto windows = engine.world().get<vve::WindowSystem>();
auto overview = windows.findWindow("overview");
if (!overview) return 2;
auto result = render.setCamera(overview->handle(),
    vve::Camera::lookAt(
        vve::Position{.value = vve::Vec3{0, 20, 1}},
        vve::Position{.value = vve::Vec3{0, 0, 0}}));
if (!result) return 3;
```

The Z offset avoids looking exactly along world up. setCamera(window,camera) overrides a view; clearCamera(window) restores the current default. Overrides are valid after init(), even before lazy GPU target creation.

Windows share the render objects and GPU scene resources. The public API does not provide per-window object visibility masks or separate scenes. Each target has its own camera, projection aspect, attachments, frame state and effect chain. Targets draw in configured window order.

Window is read-only. findWindow(id) gives runtime identity; windows() returns views. An update(world, frame, const WindowFrameData&) hook receives owning records from the latest poll.

Window condition	What a game should expect
Resize	Renderer rebuilds size-dependent resources; avoid duplicating swapchain work in game code.
Minimized / zero drawable size	Drawing can be skipped while game update steps continue.
visible(false)	The window is hidden but still renders; useful for controlled captures.
renderer("none")	The window has no render target, and per-window camera calls reject it.
Close request	The normal engine loop stops when any owned window requests close.

Startup-only public window configuration
The current facade does not expose arbitrary runtime createWindow(), setTitle() or closeWindow() calls. Plan windows at construction. Sharing a scene also means multiple views cost additional drawing, particularly shadow and post-processing work.

Source: [S36], [S29], [S37], [S03], [S14] | Maps: 76-78

Levels, removal and resource lifetimes

Give each level a record of the identities it owns, and unload those identities deliberately.

```
struct LevelState {
    std::vector<vve::RenderObjectHandle> primitives;
    std::vector<vve::RenderSceneInstanceHandle> placements;
    std::vector<vve::SceneHandle> sources;
    std::vector<vve::Entity> entities;
};
```

Use the right removal operation

Operation	What is removed or retained
removeObject(object)	One visible object. Game structs/ECS components still need your cleanup.
removeSceneInstance(instance)	The placed group's objects and its owned imported light/camera records.
removeScene(source)	Render-side source caches; rejects sources still used by live instances. AssetSystem catalog remains.
purgeUnusedAssets()	Unused CPU render meshes/materials/textures, followed by GPU synchronization as required.
clearLights()	All light lists, while leaving objects and scene instances.
clearScene()	A broad render-scene reset. Drop your associated live render handles and rebuild intended scene state.

A safe transition sequence

Stop old-level simulation/spawning. Remove placements and primitives, then their linked entities/handles. Release source caches once unused and optionally purge. Reset score, timers, game mode and views, then load the new level. Check removal results before dropping bookkeeping.

Game removal invalidates the render handle immediately. GPU allocations may remain until submitted work finishes. The engine manages retirement; your game does not wait on Vulkan fences or device idle.

Asset storage is a separate lifetime

The public AssetSystem has no unloadScene() API. Imported catalog records remain owned by it for the engine lifetime. removeScene() is a RenderSystem operation and does not erase those catalog records. For games that stream many unrelated large assets, account for this current limitation rather than assuming a render purge releases all imported memory.

> **Restart without stale references**
> After clearScene() or removals, reassign your handles from new creation results. Do not replay old IDs from a save file as live render handles. Save stable asset identifiers and gameplay data, then recreate runtime objects when loading.

Source: [S37], [S28], [S35], [S38], [S39] | Maps: 76-78

Keeping a game responsive

Keep expensive creation work out of ordinary movement, and measure what is actually slow.

Good practice	Reason in this implementation
Load models and decode textures during setup	Import and upload can block; repeated file work in an update produces visible stalls.
Move objects with setObjectTransform	Reuses mesh and texture allocations; only placement changes.
Reuse source scenes and primitive shapes	Meshes/materials can be shared across visible uses.
Keep handles, not repeated geometry copies	Asset array queries return owning copies and can be expensive.
Use a modest number of shadowed lights	Point lights draw six depth views; directional lights draw four cascades.
Keep callbacks and state lifetimes clear	Avoids chasing dangling references after scene changes or GUI setup.

Resource limits you can encounter

The texture table has 64 slots. Color-space roles can require separate entries for one source. Removal/purging reclaims unused slots. Identical primitives/materials can share resources; varied appearances reduce sharing.

Position edits can detach geometry and use per-window/per-slot vertex copies. Indices/count stay fixed; normals are not recomputed. Use transforms for rigid motion.

Read diagnostics correctly

```
std::cout << "presented=" << render.renderedFrameCount()
          << " fps=" << render.renderingFramesPerSecond()
          << " windows=" << render.lastRenderedWindowCount()
          << " meshes=" << render.sceneMeshCount()
          << " textures=" << render.sceneTextureCount() << '\n';
```

Render metrics do not count simulation substeps or every acquisition attempt. Resource counts are not draw counts across instances/shadows. Time loading and updates first, then profile the GPU if drawing is slow.

Do not add your own GPU synchronization

The engine manages fences, semaphores, barriers and retirement. Facade calls need no raw Vulkan access. Captures add waits; extra windows add draws even with shared resources.

> **No public threading promise**
> This interface does not document concurrent mutation as safe. Keep facade changes and GUI/settings access on the game thread. Worker jobs can prepare independent game data, but transfer results through your own queue before making engine calls.

Source: [S37], [S13], [S14], [S47], [S38] | Maps: 76-78

Errors, captures and debugging

Check each operation, add context to errors, and use captures to inspect rendered results.

```
auto scene = assets.loadScene(path);
if (!scene) {
    std::cerr << "loadScene path=" << path
              << " error=" << vve::errorName(scene.error()) << '\n';
    return std::unexpected(scene.error());
}
```

Symptom	First things to inspect
Window opens but nothing appears	A successful instantiate/create result, active camera, clip distance, visible flags, world size and lights.
Model visible but texture missing	Resolved paths, import messages, UVs and decode fallback. Procedural texture calls can fail explicitly.
Object moves one step late	Whether game code runs before/after step rather than in a registered update hook.
Movement disappears after an ECS edit	get() returned a copy; call put() and publish the transform to RenderSystem.
missing_object / invalid_handle	Removed objects, clearScene(), wrong handle category, or a non-rendering/closed window.

Capture after the renderer exists

```
// main() excerpt: after init and scene setup, step at least once.
auto status = engine.step();
if (!status) return 1;
auto render = engine.world().get<vve::RenderSystem>();
auto captured = render.captureFrameToPng("capture.png");
if (!captured) {
    std::cerr << vve::errorName(captured.error()) << '\n';
    return 2;
}
```

Use the WindowHandle overload for a specific view. Ensure a drawable target has presented: successful step() can skip all targets. Create capture parent directories. Optional shadow readback adds waits.

Check init() and every step. GPU startup can fail after window init. Some logged per-frame failures only skip a target, so inspect counters/messages as well as expected results.

> **A useful debugging route**
> Start in game init/update and watch handles, positions and errors. Follow the source map into instantiation/drawing. CPU stepping shows recording/submission; GPU execution needs captures, validation or readback.

Source: [S42], [S46], [S37], [S26], [S14] | Maps: 76-78

The crate game, step by step

The repository game demonstrates rules, a visible world and a HUD built entirely through the facade.

Stage in examples/game/game.cpp	What the game does
main: builder, init, asset root	Create the main window, start SDL, obtain RenderSystem and resolve bundled texture paths.
local loadScene helper	Clear render content; create one repeated grass plane, a procedural sun sphere and its directional light.
camera and game state	Keep the controller, a vector of Crate records, score, spawn timer, random generator and texture selection counter.
GUI callback registration	Show score, active crates and controls. Captures refer to state alive throughout the loop.
manual loop: time and movement	Clamp its own elapsed time to 0.1 seconds, steer the camera, reset eye height and move the sun relative to the eye.
spawning and gravity	Create crates below the active limit, alternate two textures, integrate falling and clamp landed centers to y=0.5.
engine.step()	Poll current events and draw the transforms the game just submitted.
collection after drawing	Check landed crates near the camera, remove their objects and increment score.

Notice the two meanings of loadScene

The anonymous-namespace loadScene(render,root) in this game is an application helper that builds procedural objects. It is not AssetSystem::loadScene(path), which imports a model file. The game does not load a model for the ground: it calls addTexturedPlane() with half extent 20 and UV repeats 10.

Understand the manual loop's timing

The camera and falling crates are updated before engine.step(), using input from the previous poll. InputState is a live facade view, so an Escape query after step sees refreshed state. Collection occurs after the picture was drawn, so removed crates and the updated score normally become visible on the next step.

How to grow this example

Move its gameplay into a registered GameSystem update hook for fresh input before drawing. Give level state a clear owner, keep a fixed random seed for reproducible runs, and check transform/removal failures that the compact example sometimes ignores. Keep the plane and reusable texture sources; add physics only when the game rules need it.

> **Sun geometry and illumination are separate**
> The bright unlit sphere is a visual marker. Its directional light illuminates the scene. Placing the sphere at a fixed offset from the camera makes it appear distant, but this game-specific trick is not a sky system built into the engine.

Source: [S34], [S48], [S36], [S37] | Maps: 76-78

Public API task reference

Use this page to locate the public entry point for a programming task.

I want to...	Use this public API / game responsibility
Create and initialize an engine	EngineBuilder<T...>, WindowSetup, Engine::init; construction collects options.
Control execution	Engine::step or run; inspect expected errors and FrameStatus.
Find stored services/state	engine.world().get<RenderSystem/AssetSystem/WindowSystem/GuiSystem/ECS/YourSystem>().
Use fresh input and time	A registered update(world, FrameContext) hook; WindowSystem::input.
Create or load a world	RenderSystem creation methods; AssetSystem::loadScene followed by instantiateScene.
Identify an imported part	sceneInstanceObjects, objectSourceNode, AssetSystem::nodeName and hierarchy queries.
Move a visible object	setObjectTransform; retain position/rotation/scale in your game state.
Move a part with its children	keep_hierarchy instance; setSceneInstanceNodeTransform on its node.
Edit/hide/remove geometry	setObjectMeshPositions, setObjectVisible, removeObject; check each result.
Represent gameplay components	ECS create/add/get/put/view; publish components to drawing explicitly.
Move the viewpoint	Camera / DefaultCameraController and RenderSystem::setCamera.
Illuminate the scene	set/add DirectionalLight, PointLight, SpotLight; clearLights; object shadow/unlit flags.
Draw HUD or menus	GuiSystem::draw callback and Dear ImGui; keep captured state alive.
Add image effects	setPostProcessSetup and vvppl::PostProcessing; retain per-chain settings safely.
Use another window view	WindowSetup, findWindow, setCamera(WindowHandle,Camera), clearCamera.
Unload a placed model	removeSceneInstance, then removeScene/purgeUnusedAssets when no placement remains.
Inspect results	Scene counts, rendering counters/FPS, captureFrameToPng and shadow-depth samples.
Handle physics, audio or saves	Your game code/separate libraries; the current facade does not supply these services.

A practical order for a first project

Build the supplied game. Create the starter from pages 54-55. Add your desired player controls, then one interaction and a HUD. Replace a primitive with an imported model, keeping its source and placement handles. Add level ownership and restart. Profile only after a small playable loop works.

Use Part II for the resource/frame internals. This manual describes the actual inspected facade; it does not assume APIs from a larger commercial game engine.

Source: [S01], [S32], [S36], [S37], [S38], [S39], [S41], [S55] | Maps: 76-78

Source map and stepping locations

Source labels refer to this local repository snapshot; line numbers locate a useful entry point.

Repository: C:\data\GitHub\ViennaVulkanEngine
Commit: 4f0270d3739daee794a22eb304682769f67888c2
Line numbers are from the inspected snapshot. Paths below are relative to the repository.

Label	Source file : entry line
S01	src/Engine.ixx : 31
S02	src/implementations/simple.ixx : 1
S03	src/versions/simple/Engine.ixx : 23
S04	src/versions/simple/Window.ixx : 81
S05	src/versions/simple/Assets.ixx : 89
S06	src/versions/simple/Graph.ixx : 11
S07	src/versions/simple/Types.ixx : 28
S08	src/versions/simple/Render/RenderSystem.ixx : 74
S09	src/versions/simple/Render/RenderSceneImport.cpp : 214
S10	src/versions/simple/Render/RenderResources.ixx : 66
S11	src/versions/simple/Scene.ixx : 79
S12	src/versions/simple/Render/Renderer.ixx : 61
S13	src/versions/simple/Render/RendererResources.cpp : 58
S14	src/versions/simple/Render/RendererDraw.cpp : 90
S15	src/versions/simple/Render/RendererShadowPrep.cpp : 21
S16	src/versions/simple/Vulkan/Device.ixx : 46
S17	src/versions/simple/Vulkan/Memory.ixx : 45
S18	src/versions/simple/Vulkan/Resources.ixx : 48
S19	src/versions/simple/Vulkan/Pipeline.ixx : 39
S20	src/versions/simple/Vulkan/Commands.ixx : 122
S21	src/versions/simple/Vulkan/Presentation.ixx : 31
S22	src/versions/simple/Vulkan/Shadow.ixx : 21
S23	src/versions/simple/GUI.ixx : 28
S24	src/versions/simple/shaders/simple_forward.slang : 191
S25	src/versions/simple/CMakeLists.txt : 36
S26	src/versions/simple/Render/RendererDebug.cpp : 91
S27	src/versions/simple/Vulkan/Readback.ixx : 22
S28	src/versions/simple/Render/RenderSystemObjects.cpp : 21
S29	src/Types.ixx : 171
S30	src/ECSContainer.ixx : 15
S31	src/versions/simple/Vulkan/Handle.ixx : 18
S32	src/World.ixx : 30
S33	examples/sponza/sponza.cpp : 15
S34	examples/game/game.cpp : 113
S35	src/versions/simple/Render/RenderSystemScene.cpp : 50

Game guide source references

Public declarations and examples supplement the preceding implementation source map.

The manual uses the same repository snapshot as Part II. S01-S35 remain defined on page 76. The entries below identify the public declarations and practical examples used for the new recipes. Source labels in page footers refer to these maps.

Label	Source file : entry line
S36	src/Window.ixx : 131
S37	src/RenderSystem.ixx : 18
S38	src/Assets.ixx : 15
S39	src/ECSContainer.ixx : 20
S40	src/Window.cpp : 69
S41	src/Gui.ixx : 11
S42	src/Error.ixx : 20
S43	examples/postprocessing/postprocessing.cpp : 44
S44	examples/game/CMakeLists.txt : 1
S45	cmake/CMakePresets.base.json : 43
S46	.vscode/launch.json : 5
S47	src/versions/simple/shaders/simple_shared.h : 3
S48	examples/common/ExampleSupport.ixx : 8
S49	src/Math.ixx : 51

A source-reading route for game programmers

Begin with examples/game/game.cpp [S34] for the current procedural game and examples/sponza/sponza.cpp [S33] for imported content. Use Engine.ixx [S01] for hooks and ordering; public Window.ixx [S36] for input/controller; public RenderSystem.ixx [S37] for visible objects; Assets.ixx [S38] for imported-data inspection; and ECSContainer.ixx [S39] for component copy/write semantics.

For image effects, compare the setup and retained settings in the postprocessing example [S43]. For executable integration/debugging, inspect the actual example target [S44], presets [S45] and launch file [S46]. ExampleSupport [S48] is useful for the repository examples but is not part of the imported VVEngine facade.

> **How the examples were checked**
> The manual recipes were compared with public declarations and implementation behavior. The complete starter is a new illustrative source listing, not a newly compiled repository target. The names, call signatures and lifecycle explanations come from the local source; runtime behavior should be verified when you integrate it into your own game.

Repository: C:\data\GitHub\ViennaVulkanEngine
Snapshot: 4f0270d3739daee794a22eb304682769f67888c2

Facade source references

Additional entry points for typed identity, containers and facade forwarding.

All three parts use the same inspected repository commit. S50-S54 extend the preceding source maps with the public utility types and build/forwarding entry points used in Part I. S55-S58 locate the scene hierarchy added in commit 4b54ba1a.

Label	Source file : entry line
S50	src/Handle.ixx : 10
S51	src/Entity.ixx : 10
S52	src/Vector.ixx : 9
S53	src/CMakeLists.txt : 37
S54	src/RenderSystem.cpp : 54
S55	src/RenderSystem.ixx : 221
S56	src/versions/simple/Render/RenderSceneImport.cpp : 277
S57	examples/hierarchy_car/hierarchy_car.cpp : 50
S58	tests/SceneHierarchyTests.cpp : 80

A route through the facade

Start with Engine.ixx [S01], then World.ixx [S32]. Compare the selected adapter [S02] with the build selection [S53]. Read a public RenderSystem signature [S37], follow its forwarding definition [S54], then enter the implementation described in Part II.

Scope and verification

This guide documents the code in C:\data\GitHub\ViennaVulkanEngine at commit 4f0270d3739daee794a22eb304682769f67888c2. Paths and entry lines are tied to that snapshot. Engine source and build configuration were read to check the explanations. New book snippets were compared with public signatures but not compiled or run separately.

Scene hierarchy update: pages 15, 16, 21, 30, 31, 61-63, 75 and 78 were revised for commit 4b54ba1a7beaea6b66bc8d87586d51b3a4dbea52 (keep_hierarchy, setSceneInstanceNodeTransform, sceneInstanceNodeTransform and the hierarchy_car example). The new snippets on pages 62 and 63 were compile-checked with Clang. The other pages still describe 4f0270d3.

Rendering and gameplay support can evolve after this snapshot. Keep your application on the public vve interface and use the repository examples as the starting point when moving to a later revision.